Privacy-Preserving Similarity Queries for Outsourced Trajectory Data

Kelai Yi, Yuchen Su, Shiyue Huang, Yuefeng Chen, Xiong Li, *Senior Member, IEEE*, and Hongbo Liu, *Member, IEEE*

***Abstract*—Trajectory similarity query can retrieve a set of trajectories similar to the user's query from the database and is widely used in various fields such as travel recommendations. Previous studies mainly focused on accelerating trajectory similarity search in plaintext. However, with the increasing concern about privacy protection in outsourced cloud environments, conducting trajectory similarity queries while preserving privacy becomes a significant challenge. This paper proposes efficient privacy-preserving top-k and range similarity queries over trajectory data. We leverage Discrete Synchronous Euclidean Distance (DSED) to measure the spatio-temporal similarity of trajectory data, and employ a filter-then-refine strategy to enhance efficiency. Specifically, Hilbert curve-based filtering is first applied to exclude a large portion of dissimilar trajectories, followed by homomorphic encryption-based refinement to retrieve precise results. Security analysis demonstrates that our schemes protect the privacy of trajectory data, query requests, and query results. Finally, extensive experimental results indicate that the proposed methods achieve a trade-off between data availability and privacy, achieving over 99% average precision while initially filtering out 90% of dissimilar trajectories, and improving query efficiency by at least an order of magnitude.**

***Index Terms*—Trajectory similarity query, privacy preservation, cloud security, Hilbert curve, homomorphic encryption.**

I. Introduction

TRAJECTORY data similarity query is a fundamental technique for numerous trajectory-related applications, aiming to retrieve a set of trajectories similar to a given query trajectory from large-scale trajectory datasets. This task generally takes two primary forms, i.e., top-k queries and range queries. The top-k query identifies the k most similar trajectories to the input query. For example, in tourism route planning, top-k queries retrieve the k closest historical travel routes based on a user's starting point and personal preferences, thereby providing personalized travel recommendations. In contrast, range queries return all trajectories that are similar to the query trajectory within a specified range threshold. A representative case is in epidemic prevention and control, where range queries are used to identify potential close contacts by detecting trajectories that resemble those of confirmed cases, helping to curb further spread. With the rapid development of location-based services (LBS), massive trajectory data have been accumulated by service providers. Meanwhile, cloud computing has led many individuals and organizations to outsource storage and computation to cloud servers. However, trajectory data in LBS often contains sensitive user information, and executing similarity queries in outsourced environments raises significant concerns about potential information leakage.

To tackle the privacy concerns of trajectory data in the cloud computing environment, various technical approaches for trajectory privacy protection have been proposed in recent years. These include the obfuscation-based methods [2], [3], [4], [5] and cryptography-based methods [6], [7], [8]. Although obfuscation methods leveraging anonymization and differential privacy are generally more efficient, they often sacrifice the accuracy of location-based services. In contrast, cryptography-based methods offer stronger privacy protection. However, when processing large-scale trajectory data, encryption and ciphertext computations may significantly increase query processing time, and degrade overall performance.

An efficient privacy-preserving mechanism for trajectory similarity queries is essential. However, several challenges must be addressed to enable such queries. A primary difficulty lies in the measurement of trajectory similarity over encrypted data. As the trajectory similarity measures often rely on dynamic programming for trajectory alignment, performing these operations on ciphertext is challenging due to recursive comparisons and branching on encrypted values. Therefore, a well-designed similarity measure is crucial to address the complexity of operating in the ciphertext state. Additionally, another challenge is balancing query efficiency with filtering precision. To accelerate the query processing, initial-stage filtering techniques are typically employed to eliminate irrelevant trajectories while preserving privacy. However, such filtering methods may compromise querying precision. Therefore, it is necessary to develop an efficient privacy-preserving trajectory similarity query scheme with a filter-and-refine strategy.

Received 29 September 2024; revised 27 July 2025; accepted 10 September 2025. Date of publication 16 September 2025; date of current version 14 January 2026. This work was supported in part by the National Key R&D Program of China under Grant 2022YFB3103404 and in part by the National Natural Science Foundation of China under Grant 62332018. An earlier version of this paper was presented in part at the IEEE International Conference on Mobile Ad-Hoc and Sensor Systems (IEEE MASS 2023) [DOI: 10.1109/MASS58611.2023.00070]. *(Corresponding author: Xiong Li.)*

Kelai Yi is with the School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu 611731, China, and also with China Telecom, Beijing 100010, China (e-mail: yikl@chinatelecom.cn).

Yuchen Su, Shiyue Huang, Yuefeng Chen, Xiong Li, and Hongbo Liu are with the School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu 611731, China (e-mail: yu_chen_su@163.com; huangshiyue9876@gmail.com; cc.cyf1999@gmail.com; lixiong@uestc.edu.cn; hongbo.liu@uestc.edu.cn).

Digital Object Identifier 10.1109/TDSC.2025.3610093

1545-5971 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 17,2026 at 03:39:16 UTC from IEEE Xplore. Restrictions apply.

To address the aforementioned challenges, we propose two privacy-preserving top-k and range similarity queries schemes for outsourced trajectory data, and the contributions are summarized as follows:

1) We define a novel spatio-temporal similarity measure method for discrete trajectories. Additionally, we design a trajectory filtration mechanism leveraging the location-preserving property of the Hilbert curve, enabling efficient elimination of the dissimilar trajectories in a preserving privacy manner.
2) Employing a filter-and-refine strategy, we propose privacy-preserving trajectory similarity query schemes for outsourced trajectory data, wherein the cloud performs refinement of top-k and range queries using homomorphic encryption.
3) The security analysis demonstrates that our schemes can protect the data privacy of the data owner, along with the privacy of query requests and results. Furthermore, extensive experiments conducted on various trajectory datasets confirm that the proposed schemes are computationally efficient.

II. Preliminaries

In this section, we first introduce the Discrete Synchronous Euclidean Distance (DSED), which is used as the trajectory similarity measure in our schemes. We then outline the related privacy-preserving technologies, including the Hilbert curve, homomorphic encryption and proxy re-encryption.

A. Discrete Synchronous Euclidean Distance (DSED)

To evaluate spatio-temporal trajectory similarity under ciphertext, we propose DSED, an extension of STED [9]. DSED preserves meaningful similarity, especially for uniformly paced trajectories, while enabling efficient encrypted computation with a favorable accuracy-complexity trade-off.

Assume that two moving objects' trajectories, with respective lengths l_a and l_b, are represented as follows:

$$T^A = \{\tau_i^A\}_{i=1}^{l_a} = \{(t_1^A, p_1^A), (t_2^A, p_2^A), \ldots, (t_{l_a}^A, p_{l_a}^A)\},$$
$$T^B = \{\tau_j^B\}_{j=1}^{l_b} = \{(t_1^B, p_1^B), (t_2^B, p_2^B), \ldots, (t_{l_b}^B, p_{l_b}^B)\},$$

t_i^A, t_j^B are the timestamps of the trajectory records, while p_i^A, p_j^B are the coordinates of the geolocations ($i \in [1, l_a]$, $j \in [1, l_b]$). To quantify the spatio-temporal similarity between trajectories, the distance from a sampled point $\tau_i^A = (t_i^A, p_i^A)$ to trajectory T^B is computed as follows:

$$Dist(\tau_i^A, T^B) = d(p_i^A, \hat{p}_j^B) \tag{1}$$

with $\hat{p}_j^B = \frac{t_{j+1}^B - t_i^A}{t_{j+1}^B - t_i^B} \cdot p_j^B + \frac{t_i^A - t_j^B}{t_{j+1}^B - t_i^B} \cdot p_{j+1}^B$, where $d(\cdot,\cdot)$ denotes the Euclidean distance, and (t_j^B, p_j^B), (t_{j+1}^B, p_{j+1}^B) are two temporally adjacent points in T^B satisfying $t_j^B \le t_i^A < t_{j+1}^B$. Accordingly, the *synchronous pair* at point τ_i^A is represented as $\langle t_i^A : (p_i^A, \hat{p}_j^B)\rangle$.

To compute the **D**iscrete **S**ynchronous **E**uclidean **D**istance (DSED), we iterate over all the sampling points in T^A and T^B, thereby generating $l_a + l_b - 2$ synchronous pairs corresponding to the recorded timestamps. For the u-th synchronous pair, where $u \in [1, (l_a + l_b - 2)]$, the distances within every synchronous pairs is then calculated using (1), i.e.,

$$s_u = \begin{cases} Dist(\tau_u, T^A), \text{ if } \tau_u \in T^B \\ Dist(\tau_u, T^B), \text{ if } \tau_u \in T^A, \end{cases} \tag{2}$$

where τ_u denotes a sample point in trajectory T^A or T^B.

Then, the DSED between T^A and T^B is defined as:

$$\boldsymbol{DSED(T^A, T^B)} = \left(\frac{\Delta t_{1,2} \cdot s_1 + \Delta t_{h-1,h} \cdot s_h}{2} + \sum_{u=2}^{h-1} \frac{\Delta t_{u-1,u+1}}{2} \cdot s_u \right) \cdot \frac{1}{\Delta t_{1,h}}, \tag{3}$$

where $\Delta t_{a,b} = t_b - t_a$ and $h = l_a + l_b - 2$.

To illustrate the DSED similarity computation for trajectories more concretely, Fig. 1 offers an example involving two trajectories with 3 and 4 sampling points, respectively. 5 synchronous pairs are identified at timestamp $\{4, 5, 7, 9, 15\}$ in chronological order, and the distance between each pair of points is calculated as s_u, $u \in [1, 5]$. After that, the DSED value between T^A and T^B is computed using (3), i.e., $(s_1 + 3s_2 + 4s_3 + 8s_4 + 6s_5)/22$.

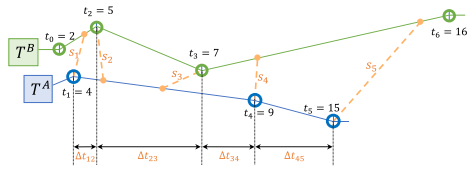


Fig. 1. Example of DSED Trajectory Similarity Calculation.

B. Hilbert Curve

The Hilbert curve [10] is a space-filling curve capable of iterating through every cell within a multi-dimensional space. Due to the ability of partially maintaining the original neighboring adjacency relationships, i.e., the locality-preserving property [11], it has been widely employed in spatial indexing of large-scale databases.

The construction of a Hilbert curve in 2-D space depends on four fundamental parameters, denoted as $\phi = \{N, \theta, P_0, \gamma\}$, where N is the curve's order (i.e., granularity of $2^N \times 2^N$), $\theta \in \{\sqcap, \sqcup, \sqsubset, \sqsupset\}$ specifies one of four possible curve orientations, $P_0 = (x_0, y_0)$ denotes the starting point, and γ denotes the scaling factor. A Hilbert curve can be constructed by recursively dividing the square region into 4 quadrants and looping through each of them. As the curve traverses each cell once, a distinctive integer ranging from $[0, 2^{2N} - 1]$ is assigned to each cell, referred to as its unique *H-value* encoding. As a result, any geolocation $p = (x, y)$ can be indexed by ϕ, which functions as a spatial encoding/decoding key [12]. Without this key, the original coordinates cannot be discerned. Based on the Hilbert curve HC^{ϕ}, the encoding algorithm $\mathbf{HIL.Enc}$ is defined as $v^{\phi} : p \to H$, where $v^{\phi}(\cdot)$ is a bijective mapping from $\mathbb{R}^2$ to $\mathbb{R}^1$, and its inverse corresponds to the decoding procedure, denoted by $\mathbf{HIL.Dec}$.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 17,2026 at 03:39:16 UTC from IEEE Xplore. Restrictions apply.

C. Homomorphic Encryption

In this part, we introduce two homomorphic encryption techniques employed to enable privacy-preserving top-k similarity queries and similarity range queries on trajectory data.

1) SHE Cryptosystem: SHE [13] is an efficient symmetric homomorphic encryption approach that supports homomorphic operations and consists of the following algorithms:

- $\mathbf{SHE.KeyGen}(k_0, k_1, k_2)$: Given a group of security parameters $\{k_0, k_1, k_2\}$ satisfying $k_1 \ll k_2 < k_0$, the SHE key generation algorithm selects two large prime numbers $\{p, q\}$ such that $|p| = |q| = k_0$. Let $\mathcal{N} = pq$, and $\mathcal{L}$ is a randomly chosen number with $|\mathcal{L}| = k_2$. Then, the algorithm outputs the public parameter $pp = (k_0, k_1, k_2, \mathcal{N})$, the secret key $sk = (p, q, \mathcal{L})$, and the basic message space $\mathcal{M} = [-2^{k_1-1}, 2^{k_1-1})$.
- $\mathbf{SHE.Enc}(sk, m)$: Given a secret key sk and a plaintext $m \in \mathcal{M}$, the SHE encryption algorithm outputs the ciphertext $E(m) = (r\mathcal{L} + m)(1 + r'p) \bmod \mathcal{N}$, where $r \in \{0, 1\}^{k_2}$ and $r' \in \{0, 1\}^{k_0}$ are two randomly chosen binary numbers.
- $\mathbf{SHE.Dec}(sk, E(m))$: Given sk and $E(m)$, the SHE decryption algorithm recovers the plaintext m' as $m' = (E(m) \bmod p) \bmod \mathcal{L} = (r\mathcal{L} + m) \bmod \mathcal{L}$. If $m' < \frac{\mathcal{L}}{2}$, we have $m \geq 0$ and $m = m'$. Otherwise, we have $m < 0$ and $m = m' - \mathcal{L}$.

Assuming that $E(m_1)$ and $E(m_2)$ are the corresponding ciphertexts of m_1 and m_2, respectively, SHE satisfies the following homomorphic properties:

- *Homomorphic addition-I:* $E(m_1) + E(m_2) \bmod \mathcal{N} \rightarrow E(m_1 + m_2)$.
- *Homomorphic multiplication-I:* $E(m_1) \cdot E(m_2) \bmod \mathcal{N} \rightarrow E(m_1 \cdot m_2)$.
- *Homomorphic addition-II:* $E(m_1) + m_2 \bmod \mathcal{N} \rightarrow E(m_1 + m_2)$.
- *Homomorphic multiplication-II:* $E(m_1) \cdot m_2 \bmod \mathcal{N} \rightarrow E(m_1 \cdot m_2)$ when $m_2 > 0$.

In the public key setting of SHE as described in [14], i.e., $pk = \{E(0)_1, E(0)_2, pp\}$, the encryption of a message $m \in \mathcal{M}$ can be achieved by $E(m) = m + r_1 \cdot E(0)_1 + r_2 \cdot E(0)_2 \bmod \mathcal{N}$, where $E(0)_i$ represents the ciphertext of 0, and r_i is a k_2-bit random number, $i = 1, 2$.

2) CKKS Cryptosystem: The CKKS (Cheon-Kim-Kim-Song) [15] is a homomorphic encryption scheme based on the Ring Learning with Error (RLWE) problem [16], achieving semantic security. Compared with other homomorphic encryption techniques, CKKS allows approximate arithmetic operations on encrypted real (complex) numbers without the need for decryption, and is widely used in various fields that involve floating-point computations, such as finance, healthcare, and cloud computing. Specifically, the CKKS includes the following algorithms:

- $\mathbf{CKKS.KeyGen}(n, q)$: Given a positive integer n and a modulus q, the CKKS key generation algorithm selects a random polynomial a, a secret polynomial$s$, and a noise polynomial e from the polynomial ring $R_q = \frac{\mathbb{Z}_q[X]}{X^n+1}$. It then outputs a secret key $sk = s$, a public key $pk = (-as + e, a)$ and an evaluation key evk.
- $\mathbf{CKKS.Enc}(p, pk)$: Given a plaintext $p \in R_q$ and a public key $pk = (b, a)$, the CKKS encryption algorithm outputs the ciphertext $\hat{c} = (p + b, a)$.
- $\mathbf{CKKS.Dec}(\hat{c}, sk)$: Given a ciphertext $\hat{c} = (c_0, c_1) \in R_q^2$ and a private key $sk = s$, the CKKS decryption algorithm recovers the plaintext $p = c_0 + c_1\ s$.
- $\mathbf{CKKS.Add}(\hat{c}, \hat{c}')$: Given two ciphertexts $\hat{c} = (c_0, c_1)$ and $\hat{c}' = (c'_0, c'_1)$, the CKKS homomorphic addition algorithm returns the ciphertext $(c_0 + c'_0, c_1 + c'_1)$.
- $\mathbf{CKKS.Mult}(\hat{c}, \hat{c}')$: Given ciphertexts $\hat{c} = (c_0, c_1)$ and $\hat{c}' = (c'_0, c'_1)$, the CKKS homomorphic multiplication algorithm outputs the ciphertext $(c_0c'_0, c_0c'_1 + c_1c'_0, c_1c'_1)$.

D. Proxy Re-Encryption

Proxy re-encryption (PRE) [17] is a special type of public key encryption that enables a proxy to transform a ciphertext encrypted with the delegator i's key into a ciphertext of the identical message utilizing the delegatee j's key, without the proxy gaining access to the content of the original message. Commonly, a PRE scheme includes the following algorithms:

- $\mathbf{PRE.GenKey}(k_i)$: Given a security parameter k_i, the PRE key generation algorithm outputs a public/private key pair for user i, denoted as $\{sk_i, pk_i\}$.
- $\mathbf{PRE.ReKey}(\{sk_i, pk_i\}, \{sk_j, pk_j\})$: Given two key pairs $\{sk_i, pk_i\}$ and $\{sk_j, pk_j\}$, where sk_j is optional, the PRE re-encryption key generation algorithm outputs a re-encryption key rk_{ij}.
- $\mathbf{PRE.Enc}(pk_i, m)$: Given a public key pk_i and a message m, the PRE encryption algorithm outputs a ciphertext c_i of user i.
- $\mathbf{PRE.ReEnc}(rk_{ij}, c_i)$: Given a re-encryption key rk_{ij} and a ciphertext c_i of user i, the PRE re-encryption algorithm outputs a re-encrypted ciphertext c_j for user j.
- $\mathbf{PRE.Dec}(sk_j, c_j)$: Given a secret key sk_j and a ciphertext c_j, the PRE decryption algorithm outputs a plaintext m.

III. Problems, Models and Design Goals

This section first provides formal definitions of the trajectory similarity query problems. Then, we present the system model, security model, and the design goals.

A. Query Definitions

1) Privacy-Preserving Top-k Trajectory Similarity Query (PTTSQ): Given a trajectory dataset $\mathcal{T}$ containing n trajectories with associated identity information, along with a query trajectory Q and an integer value k, the PTTSQ aims to retrieve the k trajectories most similar to Q, including their identity and similarity values, while preserving trajectory privacy. Specifically, the goal is to identify a subset $\mathcal{R} \subseteq \mathcal{T}$ such that $|\mathcal{R}| = k$ and for every $R \in \mathcal{R}$ and $\forall T \notin \mathcal{R}$, it holds that $DSED(Q, R) \leq DSED(Q, T)$.

2) Privacy-Preserving Trajectory Similarity Range Query (PTSRQ): Given a trajectory dataset $\mathcal{T}$, along with a query trajectory Q and a distance threshold δ, the PTSRQ aims to retrieve all trajectories in $\mathcal{T}$ whose distances to Q do not exceed

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 17,2026 at 03:39:16 UTC from IEEE Xplore. Restrictions apply.

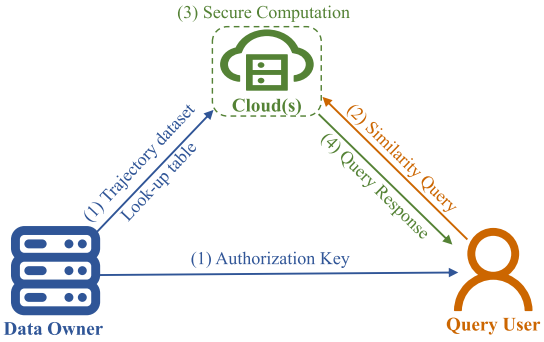


Fig. 2. System Model.

δ, while preserving privacy. That is, it identifies a subset $\mathcal{R} \subseteq \mathcal{T}$, such that for every $R \in \mathcal{R}$, $DESD(Q, R) \leq \delta$ holds.

B. System Model

This work adopts a cloud-assisted model for similarity query over trajectory data, involving three types of entities: the data owner DO, the query user QU, and the cloud server(s) CS, as depicted in Fig. 2.

- *Data Owner DO:* DO possesses a dataset containing n trajectories, denoted as $\mathcal{T} = \{\langle ID_i, T^i \rangle\}_{i=1}^{n}$, where T^i is the i-th trajectory, and ID_i denotes the identity information of the corresponding moving object. The identity information often includes sensitive personal details such as an individual's name, ID number, and contact number, which necessitate protection. To save computation and storage expenses, DO outsources the trajectory similarity search services to CS(s). For privacy concerns, DO encrypts the dataset $\mathcal{T}$ prior to outsourcing. Additionally, a Hilbert-based look-up table is constructed to facilitate the privacy-preserving search over CS(s).

- *Query User QU:* QU denotes a user authorized to issue similarity queries on DO's trajectory data. QU can initiate either a top-k query request $\{Q, k\}$ or a range query request $\{Q, \delta\}$ to the cloud server, and obtain the queried trajectory results $\mathcal{R}$ while preserving privacy, with their DSED similarity values $D_j = DSED(R^j, Q)$ satisfying the corresponding querying requirement.

- *Cloud Server(s) CS:* The trajectory data of DO are encrypted and outsourced to CS along with the Hilbert-based look-up table. Upon receiving a query request from QU, CS first filters the large-scale trajectory dataset to identify the potentially similar trajectories. It then calculates the actual result trajectories in a privacy-preserving manner and returns the query results to QU. To enable secure similarity computation in the cloud, this paper explores both single-cloud and dual-cloud server models, involving one and two cloud servers, respectively.

As discussed in our previous version published in IEEE MASS 2023 [1], the dual-cloud server model under a non-collusion assumption demonstrates notable computational efficiency. In this architecture, $CS = \{CS_1, CS_2\}$, where CS_1 stores encrypted data along with a Hilbert-based look-up table, and CS_2 holds the secret key for subsequent operations. Despite its performance advantages, this model may introduce increased system complexity and higher maintenance costs. In contrast, although privacy-preserving computation on a single cloud server is typically less efficient, our study also investigates a single-cloud solution that supports similarity range queries in a simple and cost-effective manner, reducing both communication overhead and maintenance burden.

C. Security Model

In our model, all three entities are assumed to be *semi-honest*, i.e., they follow the protocol honestly but attempt to infer private information about others. The dual-cloud framework further presumes that CS_1 and CS_2 do not collude with each other, which is a reasonable assumption given the competitive nature of cloud service providers [18]. Additionally, DO and QU are supposed not collude with CS regarding the privacy of their data. As semi-honest participants, CS may execute cloud inference attacks to acquire the plaintext of trajectory data, as well as the query requests and results. Similarly, both QU and DO may also have an interest in disclosing each other's trajectory data privacy.

D. Design Goals

In this work, we aim to achieve privacy-preserving schemes for trajectory similarity queries, and the specific goals are detailed as follows:

1) Query Precision: The filtration technique is utilized to reduce computational overhead in similarity calculations over large-scale trajectory data, which may potentially compromise the precision of the similarity query. Therefore, it is crucial to ensure the precision of the retrieved query results, i.e., $pr = \frac{|R \cap C|}{|C|}$, where R and C denote the final retrieved results and the precise set of similar trajectories, respectively.

2) Privacy Preservation: Under the defined security model where DO, QU and CS are all semi-honest entities, the proposed trajectory similarity query schemes should achieve the following security goals.

- *Data Privacy:* The location information and the identities of trajectory dataset $\mathcal{T}$ cannot be revealed by CS. Meanwhile, while QU can retrieve the query result $\mathcal{R}$, it gains no additional information regarding $\mathcal{T} - \mathcal{R}$.
- *Query Request Privacy:* The location privacy of query trajectory Q should be preserved against DO and CS.
- *Query Result Privacy:* The query results, including the identities and trajectories in the set $\mathcal{R}$, must be protected against both DO and CS.

3) Efficiency: Performing similarity queries over encrypted trajectory data may significantly increase the computational costs and communication overheads within the system. Therefore, the proposed schemes must be optimized to ensure efficient execution of similarity search operations.

IV. Our Proposed Schemes

This section first introduces a Hilbert-curve-based distance approximation algorithm for trajectory filtration. We then present our privacy-preserving top-k trajectory similarity query and range query schemes, PTTSQ and PTSRQ.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 17,2026 at 03:39:16 UTC from IEEE Xplore. Restrictions apply.

A. Hilbert-Curve-Based Distance Approximation Algorithm for Trajectory Filtration

To enhance the efficiency of privacy-preserving trajectory similarity queries, we design a distance approximation algorithm based on the locality-preserving property of the Hilbert curve [11]. To filter out the dissimilar trajectories over large-scale data more precisely, the algorithm is progressively refined using a cluster of Hilbert curves and a series of pivot points, forming the foundation of our privacy-preserving trajectory filtration technique.

1) Intuitive Hilbert-Curve-Based Distance (HD): Given a single Hilbert curve HC^{ϕ} with parameter ϕ, the proximity between two points can be intuitively estimated by the difference in their H-values, referred to as the **H**ilbert-curve-based **D**istance (HD), i.e.,

$$d_H(p_i, p_j) = |v^{\phi}(p_i) - v^{\phi}(p_j)| = |H_i^{\phi} - H_j^{\phi}|, \quad (4)$$

where p_i, p_j are two coordinate points in a multi-dimensional space, and H_i^{ϕ}, H_j^{ϕ} refer to their respective H-values on HC^{ϕ}, computed using the encoding rule $v^{\phi}(\cdot)$.

Due to the locality-preserving property of the Hilbert curve, the adjacent coordinate points in a multi-dimensional space usually have similar H-values after encoding, while the coordinate points that are far apart are unlikely to have close encoded H-values. This enables the early elimination of many dissimilar trajectories through preliminary filtering. However, distance approximation may sometimes be inaccurate when two coordinate points have significantly different H-values but are spatially close, leading to the erroneous exclusion of truly similar trajectories from the candidate set. This necessitates further refinement.

2) Hilbert-Curve-Based Distance Modified by Transformation (HD_t): For a 2-D geo-location grid of of size $2^N \times 2^N$, the generated N-order Hilbert curve can be divided into four sub-squares. Points within each sub-square are encoded into one of the following intervals: $\{[0, 2^{2N-2}), [2^{2N-2}, 2^{2N-1}), [2^{2N-1}, 3 \cdot 2^{2N-2}), [3 \cdot 2^{2N-2}, 2^{2N})\}$, and may exhibit similar H-values. However, for points located in different sub-squares, encoding them using a single Hilbert curve HC^{ϕ} may result in large differences between their H-values, even if they are very close in the 2-D space. This discrepancy is not tolerable for similarity queries, as the true relevant results may be mistakenly excluded during filtration. To overcome the limitations mentioned above, we propose a modified Hilbert-curve-based distance algorithm, improving upon the intuitive distance in Equation (4) by incorporating multiple Hilbert curves. This approach first generates several different Hilbert curves from an original Hilbert curve through space transformation, then combines them to mitigate the shortcomings of using a single curve, enhancing filtration precision.

Let the parameters of a Hilbert curve be denoted by $\phi = \{N, \theta, P_0, \gamma\}$, where N represents the Hilbert curve's order (i.e., granularity), θ specifies the curve's orientation, $P_0 = (x_0, y_0)$ denotes the starting point, and γ denotes the scale factor [19], [20]. By varying parameters θ and P_0 in ϕ, the original curve HC^{ϕ} can be rotated and translated, thus generating a cluster of different Hilbert curves derived from HC^{ϕ}. This cluster, sharing a common scale γ, is represented as $\mathcal{HC} = \{HC^{\phi}; HC^{I}, HC^{II}, \ldots, HC^{r}\}$, with corresponding encoding rules $\mathcal{V} = \{v^{\phi}; v^{I}, v^{II}, \ldots, v^{r}\}$. For any two coordinate points p_i and p_j, their Hilbert-curve-based Distance modified by Transformation (HD_t) is defined as the minimum Hilbert distance computed over all curves in the cluster, i.e.,

$$d_{\mathcal{H}}(p_i, p_j) = \min_{\eta \in \varphi} |v^{\eta}(p_i) - v^{\eta}(p_j)| = \min_{\eta \in \varphi} |H_i^{\eta} - H_j^{\eta}|, \quad (5)$$

where the index set $\varphi = \{\phi, I, II, \ldots, r\}$, and H_i^{η} and H_j^{η} represent the H-values of p_i and p_j on HC^{η}, computed according to the encoding rule $v^{\eta}(\cdot)$. It is worth noting that, for a fixed scale γ on a Hilbert curve, the mapping scope can be enlarged by increasing the curve's order N without altering its scale. This fundamental property enables our proposed Hilbert-curve-based approaches to inherently support both spatial expansion and dynamic updates of trajectory data points. Specifically, when new trajectory points exceed the current spatial coverage, only these new points need to be re-encoded using the Hilbert curve, while previously encoded values remain unchanged. As a result, spatial scalability is achieved without recoding the existing data, and the locality-preserving properties are maintained across the expanded space.

3) Hilbert-Curve-Based Distance Modified by Pivot-Point (HD_p): To enhance the precision of privacy-preserving trajectory filtering, the Hilbert-curve-based distance algorithm is progressively refined by introducing some pivot points, allowing for a more precise approximation of trajectory similarity.

Specifically, for any two coordinate points p_i and p_j, the algorithm randomly selects a set of pivot points from the union of their neighborhoods, i.e., $\mathrm{N}(p_i) \cup \mathrm{N}(p_j)$, denoted as $\mathcal{W} = \{w_1, \ldots, w_m\}$. Given a cluster of $r+1$ Hilbert curves as defined earlier, $\mathcal{HC} = \{HC^{\phi}; HC^{I}, HC^{II}, \ldots, HC^{r}\}$, the algorithm seek to minimize the cumulative $\mathrm{HD_t}$ values over a sequence of subpaths forming a reachable path from p_i to p_j through a subset of $\mathcal{W}$. The minimum such value is referred to as the Hilbert-curve-based Distance modified by Pivot-point ($\mathrm{HD_p}$),

$$d_{\mathcal{H}^*}(p_i, p_j) = \min_{w_i \in W} \left\{ d_{\mathcal{H}}(p_i, w_1) + d_{\mathcal{H}}(w_m, p_j) + \sum_{i=1}^{m-1} d_{\mathcal{H}}(w_i, w_{i+1}) \right\}, \quad (6)$$

where $d_{\mathcal{H}}$ is defined in Equation (5).

For a better understanding of the modified Hilbert-curve-based distance, a simple example involving three Hilbert curves $\mathcal{HC} = \{HC^{\phi}, HC^{I}, HC^{II}\}$ is provided in Fig. 3, where HC^{I} and HC^{II} are are derived from HC^{ϕ} via rotation and translation, respectively, with associated parameters $\{\theta^{\phi}, \theta^{I}, \theta^{II}\} = \{\sqcap, \sqsupset, \sqcap\}$, $\{P_0^{\phi}, P_0^{I}, P_0^{II}\} = \{(0,0), (0,0), (-3,-3)\}$. Given the points $p_1 = (5,2)$ and $p_2 = (2,4)$, their corresponding H-values under the three Hilbert curves are $H_1^{\phi} = 55$, $H_1^{I} = 34$, $H_1^{II} = 49$, $H_2^{\phi} = 30$, $H_2^{I} = 9$, $H_2^{II} = 32$, respectively. Accordingly, the standard HD distance under HC^{ϕ} is computed as $d_H(p_1, p_2) = |H_1^{\phi} - H_2^{\phi}| = |55 - 30| = 25$. Incorporating transformation, the modified $\mathrm{HD_t}$ distance is calculated as $d_{\mathcal{H}}(p_1, p_2) = \min_{\eta \in \{\phi, I, II\}} |H_1^{\eta} - H_2^{\eta}| =$

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 17,2026 at 03:39:16 UTC from IEEE Xplore. Restrictions apply.

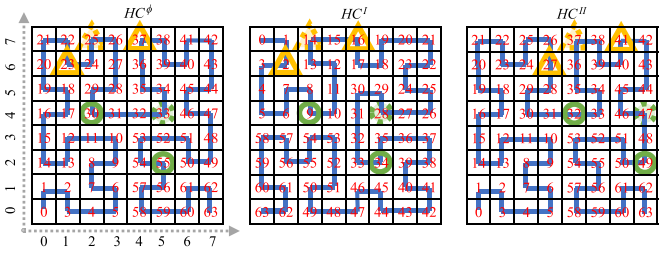


Fig. 3. An example of modified Hilbert-curve-based distance.

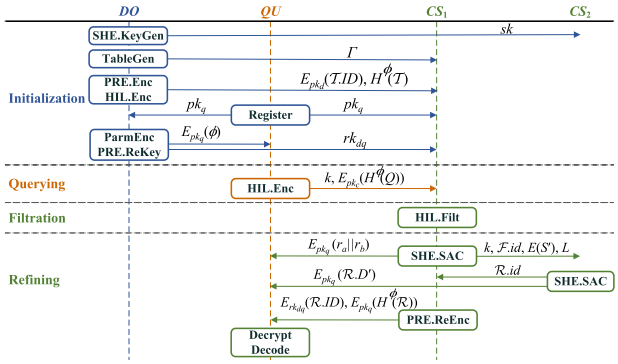


Fig. 4. System Flow Diagram for PTTSQ.

$\min\{25, 25, 17\} = 17$. Furthermore, by introducing a pivot-point $w_1 = (5, 4)$, the pivot-based distance HD_p is given by $d_{\mathcal{H}^*}(p_1, p_2) = \min\{d_{\mathcal{H}}(p_1, w_1) + d_{\mathcal{H}}(w_1, p_2)\} = \min\{3 + 2\} = 5$. Similarly, for points $p_3 = (1, 6)$ and $p_4 = (4, 7)$, we have the HD distance $d_H(p_3, p_4) = |H_3^{\phi} - H_4^{\phi}| = |23 - 37| = 14$, and the HD_t distance $d_{\mathcal{H}}(p_3, p_4) = \min_{\eta \in \{\phi, I, II\}} |H_3^{\eta} - H_4^{\eta}| = \min\{14, 14, 14\} = 14$. If $w_2 = (2, 7)$ is chosen as the pivot-point, the HD_p distance would be $d_{\mathcal{H}^*}(p_3, p_4) = \min\{d_{\mathcal{H}}(p_3, w_2) + d_{\mathcal{H}}(w_2, p_4)\} = \min\{2 + 2\} = 4$.

B. Description of PTTSQ: A Privacy-Preserving Top-k Trajectory Similarity Query Scheme

To improve the search efficiency, PTTSQ adopts a filter-and-refine paradigm [21], and consists of five stages, i.e., initialization, querying, preliminary filtration, refinement, and response, as depicted in Fig. 4. In this framework, QU initiates a top-k query request to DO and the two cloud servers (CS_1 and CS_2) for the query service. Utilizing the Hilbert-curve-based filtration, the CS_1 preliminarily excludes the majority of dissimilar trajectories, retaining only K candidate trajectories ($k \leq K \ll n$) for further refinement, which ultimately yields the final top-k results.

1) Initialization: To initialize the system, DO generates a cluster of Hilbert curves $\mathcal{HC}$ and a public-secret key pair $\{pk, sk\}$ for SHE encryption, followed by the construction of a lookup table. After encoding the trajectory data using Hilbert curves, DO outsources the encrypted trajectory data to CS_1. When a QU registers, DO generates a re-encryption key rk_{dq}, and provides the encrypted parameters associated with the chosen Hilbert curve. The initialization procedure is detailed below.

TABLE I
AN EXAMPLE OF THE HILBERT-BASED LOOK-UP TABLE

H_{xy}^{ϕ}	H_{xy}^{I}	H_{xy}^{II}	$\cdots$	H_{xy}^{r}	$(E(x), E(y))$
000000	010101	101010	$\cdots$	111111	$(E(0), E(0))$
000001	010110	101011	$\cdots$	111100	$(E(0), E(1))$
000010	010111	101000	$\cdots$	111101	$(E(1), E(1))$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
111111	000000	010101	$\cdots$	101010	$(E(7), E(7))$

Step 1. Data Pre-processing: DO first applies the Douglas-Peucker (DP) algorithm [22] for data thinning, thereby extracting typical point sequences from the trajectory data. Trajectory points are are assumed to be situated within cells of a $2^N * 2^N$ square grid. The pre-processed dataset of n trajectories is denoted as $\mathcal{T} = \{\langle ID_i, T^i \rangle\}_{i=1}^{n}$, where ID_i represents the identity of a moving object and $T^i = \{(t_1^i, p_1^i), (t_2^i, p_2^i), \ldots, (t_{l_i}^i, p_{l_i}^i)\}$ represents the i-th trajectory in $\mathcal{T}$.

Step 2. Hilbert-based Lookup-table Construction: DO constructs a Hilbert-based Lookup-table Γ by running **TableGen** through the following procedure. It first selects a Hilbert curve parameter ϕ, then creates a cluster of Hilbert curves by varying the orientation θ and starting point P_0, resulting in $\mathcal{HC} = \{HC^{\phi}; HC^{I}, HC^{II}, \ldots, HC^{r}\}$, where r represents the number of modified curves. For each HC^{η}, where $\eta \in \{\phi, I, II, \ldots, r\}$, DO applies $\mathbf{HIL.Enc}((x, y), \phi^{\eta})$ to every coordinate point (x, y) in the target space to obtain the corresponding H-values H_{xy}^{η}, where $x, y \in [0, 2^N - 1]$. Simultaneously, DO invokes $\mathbf{SHE.KeyGen}$ to generate a public/secret key pair $\{pk, sk\}$, and encrypts the 2-D coordinates as $(E(x), E(y))$. Finally, DO builds the lookup-table Γ by associating the encrypted 2-D coordinates with the relevant H-values under various Hilbert curves, formally represented as $\Gamma = \langle H_{xy}^{\phi} : [H_{xy}^{I}, H_{xy}^{II}, \ldots, H_{xy}^{r}, (E(x), E(y))] \rangle$, where H_{xy}^{ϕ} serves as the key for subsequent data retrieval. An example of the Hilbert-based look-up table is shown in Table I.

Step 3. Trajectory Encoding and Encryption: The trajectory encoding is performed by DO through $\mathbf{HIL.Enc}(p_j^i, \phi)$, which processes each coordinate point p_j^i based on the specified parameter ϕ, i.e., $H^{\phi}(\mathcal{T}) = \{H^{\phi}(T^i) = [(t_1^i, H^{\phi}(p_1^i)), (t_2^i, H^{\phi}(p_2^i)), \ldots, (t_{l_i}^i, H^{\phi}(p_{l_i}^i))] | i = 1, 2, \ldots, n\}$. Additionally, DO performs $\mathbf{PRE.Enc}(pk_d, ID)$ using the public key of PRE and generates the encrypted trajectory identities $E_{pk_d}(\mathcal{T}.ID) = \{E_{pk_d}(ID_i) | i = 1, 2, \ldots, n\}$.

At last, DO outsources the lookup-table Γ, the encoded trajectories $H^{\phi}(\mathcal{T})$ and the encrypted identities $E_{pk_d}(\mathcal{T}.ID)$ to CS_1. Meanwhile, DO provides CS_2 with the sk.

Step 4. Query User Registration: To register with the system and query DO's trajectory data, QU submits its public key pk_q for both DO and CS_1. Then, DO encrypts the chosen curve parameter ϕ using pk_q and executes $\mathbf{PRE.ReKey}(sk_d, pk_d, pk_q)$ to create a re-encryption key rk_{dq} for QU. Finally, DO sends $E_{pk_q}(\phi)$ to QU and rk_{dq} to CS_1.

2) Querying: Assume that QU's query trajectory is $Q = [(t_1, q_1), (t_2, q_2), \ldots, (t_{l_0}, q_{l_0})]$. For each point in

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 17,2026 at 03:39:16 UTC from IEEE Xplore. Restrictions apply.

the trajectory, QU applies $\mathbf{HIL.Enc}(q_j, \phi)$, producing a sequence of H^ϕ-values denoted by $H^\phi(Q) = [(t_1, H^\phi(q_1), (t_2, H^\phi(q_2), \ldots, (t_{l_0}, H^\phi(q_{l_0}))]$. Subsequently, QU encrypts $H^\phi(Q)$ using the public key of CS_1 and sends the query request $\{E_{pk_c}(H^\phi(Q)), k\}$ to CS_1, where k is an integer.

3) Preliminary Filtration: During the filtration stage, CS_1 performs a top-k similarity query without prior knowledge, using the Hilbert-based lookup-table Γ to retrieve K potential trajectories as the initial candidate results $\mathcal{F}$.

Step 1. Hilbert-curve-based Similarity Calculation: After receiving $H^\phi(Q)$, CS_1 refers to the look-up table Γ and calculates the trajectory similarity between the query trajectory Q and each trajectory $T^i \in \mathcal{T}$. This yields a similarity set $D^{\mathcal{H}} = \{D_i^{\mathcal{H}} = DSED_{\mathcal{H}}(Q, T^i) | (i = 1, 2, \ldots, n)\}$, where $DSED_{\mathcal{H}}$ denotes the approximation of DSED based on the modified Hilbert-curve-based distance, as previously described in Section II-A. Specifically, for each trajectory point $\tau_u \in Q \cup T^i$, $u = \{1, 2, \ldots, l_0 + l_i - 2\}$, the distance between $\tau_u = (t_u, p_u)$ and the other trajectory T^i or Q is calculated as

$$s_u = \begin{cases} Dist((t_u, p_u), T^i), \text{ if } (t_u, p_u) \in Q \\ Dist((t_u, p_u), Q) \text{ , if } (t_u, p_u) \in T^i, \end{cases} \tag{7}$$

where the distance between two spatial points is measured by the modified Hilbert-curve-based distance. Thus, for each trajectory $T^i \in \mathcal{T}$ $(i = 1, 2, \ldots, n)$, the Hilbert-curve-based trajectory similarity is computed as $DSED_{\mathcal{H}}(Q, T^i) = (\frac{\Delta t_{1,2} \cdot s_1 + \Delta t_{h-1,h} \cdot s_h}{2} + \sum_{u=2}^{h-1} \frac{\Delta t_{u-1,u+1}}{2} \cdot s_u) \cdot \frac{1}{\Delta t_{1,h}}$, where $\Delta t_{a,b} = t_b - t_a$, $h = l_0 + l_i - 2$, and l_0 and l_i denote the trajectory lengths of Q and T^i, respectively.

Step 2. Trajectory Filtration: CS_1 sorts the trajectory dataset according to the similarity values $D^{\mathcal{H}}$ and eliminates trajectories dissimilar to the query. It then selects a top-K candidate set $\mathcal{F}$ from the total n trajectories in DO as the initial filtration results, where $|\mathcal{F}| = K \ll n$. For subsequent refinement, CS_1 stores the indices of these K candidates as $\mathcal{F}.id$.

4) Refinement: To refine the candidate set of trajectories, CS_1 and CS_2 execute a secure protocol over the encrypted trajectory data to identify the exact top-k results. Finally, QU retrieves the top-k query results, including their similarities and identities, through decoding and decryption algorithms.

Step 1. Encrypted Trajectory Retrieval: To retrieve the encrypted trajectory data in $\mathcal{F}$ for the further refinement, CS_1 searches for the encrypted query trajectory's coordinates according to their H^ϕ-values in the look-up table Γ, and the obtained SHE ciphertexts can be represented as $E(Q) = [(t_1, E(q_1)), (t_2, E(q_2)), \ldots, (t_{l_0}, E(q_{l_0}))]$. Similarly, a set of K encrypted trajectories can be acquired by looking up the SHE ciphertexts of the candidate trajectories' coordinates, expressed as $E(\mathcal{F}) = \{E(T_f^j) = [(t_1^j, E(p_1^j)), (t_2^j, E(p_2^j)), \cdots, (t_{l_j}^j, E(p_{l_j}^j))] | j \in \mathcal{F}.id\}$.

Step 2. Homomorphic Calculation of Exact Distances: After obtaining the SHE ciphertexts $E(\mathcal{F})$ and $E(Q)$, CS_1 calculates the encrypted distance s_u in (7) between Q and each $T_f^j \in \mathcal{F}$ using homomorphic operations over the SHE ciphertexts, resulting in $E(s_u^{(j)})$, where $j \in [1, K]$, $u \in [1, l_0 + l_j - 2]$, and l_0 and l_j denote the lengths of trajectories Q and T_f^j, respectively. Then, for each $T_f^j \in \mathcal{F}$, the total sum of synchronous distances at all timestamps can be calculated in the ciphertext state as

$$E(S_j) = \Delta t_{1,2} \cdot E(s_1^{(j)}) + \Delta t_{h_j - 1, h_j} \cdot E(s_{h_j}^{(j)}) + \sum_{u=2}^{h_j - 1} \Delta t_{u-1,u+1} \cdot E(s_u^{(j)}), \tag{8}$$

where $h_j = l_0 + l_j - 2$. Let L_j represent the total time interval $\Delta t_{1,h_j}$, the similarity value can be expressed as $DSED(Q, T_f^j) = S_j/(2L_j)$. However, despite the homomorphic properties of SHE, it does not support division and comparison operations in ciphertext state. Therefore, the Secure Average Comparison protocol ($\mathbf{SHE.SAC}$) is proposed to refine the exact top-k similar trajectories from $\mathcal{F}$.

Step 3. Secure and Exact Top-k Search: To retrieve the top-k similar trajectories $\mathcal{R}$ from the set of candidate trajectories $\mathcal{F}$, the dual cloud servers execute the $\mathbf{SHE.SAC}$ protocol without collusion, and obtain the indexes of top-k trajectories with the smallest DSED values, as shown in lines 1-13 of Algorithm 1.

In this process, CS_1 first selects two random numbers, $r_a \in (0, 2^{k_1})$ and $r_b \in (-2^{k_1}, 2^{k_1})$, and transmits $E_{pk_q}(r_a \| r_b)$ to QU. It then calculates $\{E(S_j') = r_a \cdot E(S_j) + r_b \cdot L_j | j = 1, 2, \ldots, K\}$ and forwards these values along with the corresponding L_j and id_j to CS_2. Upon receiving $\{\langle E(S_j'), L_j, id_j \rangle\}_{j=1}^K$, CS_2 decrypts S_j' using sk and further computes $\bar{D}_j' = S_j'/L_j$ $(j = 1, 2, \ldots, K)$.

Although the DSED values are randomly perturbed by r_a and r_b, the exact ranking of each candidate trajectory in $\mathcal{F}$ remains the same as the order ranked by $\bar{D}_j'$. Thus, the DSED values of the top-K results are preserved in sorted order after filtration. Hence, CS_2 can directly apply the heapsort algorithm to D_j' to obtain the exact top-k results $\mathcal{R}.D' = \{\bar{D}_v' | v = 1, 2, \ldots, k\}$ along with their corresponding indexes, denoted as $\mathcal{R}.id$. Subsequently, CS_2 sends $E_{pk_q}(\mathcal{R}.D')$ to QU and transmits $\mathcal{R}.id$ to CS_1.

5) Response: Finally, CS_1 performs proxy re-encryption on the refined identity information and returns the encrypted top-k query results to QU. To retrieve the results, QU decrypts and decodes the data to recover the top-k trajectories, similarity values, and identity information. The query response process mainly includes the following steps:

Step 1. Proxy Re-encryption of Query Result: Upon receiving the indexes $\mathcal{R}.id$, CS_1 retrieves the corresponding k encoded trajectories $H^\phi(\mathcal{R})$ and their encrypted identity information $E_{pk_d}(\mathcal{R}.ID)$ from $H^\phi(\mathcal{T})$ and $E_{pk_d}(\mathcal{T}.ID)\rangle$, respectively. It then re-encrypts $\mathcal{R}.ID$ by executing $\mathbf{PRE.ReEnc}(rk_{dq}, \mathcal{R}.ID)$ using the proxy re-encryption key rk_{dq}. Finally, CS_1 transmits the encrypted trajectory data $\{E_{rk_{dq}}(\mathcal{R}.ID), E_{pk_q}(H^\phi(\mathcal{R}))\}$ to QU.

Step 2. Query Result Recovery: At last, after receiving $\{E_{rk_{dq}}(\mathcal{R}.ID), E_{pk_q}(H^\phi(\mathcal{R})), E_{pk_q}(r_a \| r_b)\}$ from CS_1 and $E_{pk_q}(\mathcal{R}.D')$ from CS_2, QU retrieves the query results comprising identities, trajectories and DSED values, denoted as $\mathcal{R} = \{\langle ID_j, R^j, D_j \rangle\}_{j=1}^k$. Specifically, to obtain the top-k trajectories, QU utilizes $\mathbf{HIL.Dec}(\phi, H^\phi(\mathcal{R}))$ to recover R^j.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 17,2026 at 03:39:16 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: **S**ecure **A**verage **C**omparison Protocol.

Input: CS_1 holds the encrypted summed distances, total time intervals, and the indexes of K trajectories in $\mathcal{F}$, i.e., $\{\langle E(S_j), L_j, id_j\rangle\}_{j=1}^{K}$; CS_2 holds the secret key sk;

Output: The smallest k averaged distances $\mathcal{R}.D = \{\bar{D}_v = S_v/L_v \,|\, v = 1, 2, \cdots, k\}$ and their indexes $\mathcal{R}.id$.

1 (CS_1)
2 randomly choose $r_a \in (0, 2^{k_1}), r_b \in (-2^{k_1}, 2^{k_1})$;
3 **for** $j = 1$ *to* K **do**
4 compute $E(S'_j) = r_a \cdot E(S_j) + r_b \cdot L_j$;
5 use pk_q to encrypt $r_a \| r_b$ and send $E_{pk_q}(r_a \| r_b)$ to QU;
6 send $\{\langle E(S'_j), L_j, id_j\rangle\}_{j=1}^{K}$ to CS_2;
7 (CS_2)
8 use sk to recover S'_j;
9 **for** $j = 1$ *to* K **do**
10 compute $\bar{D}'_j = S'_j/L_j$;
11 do heap sort on $\{\bar{D}'_j\}_{j=1}^{K}$ in an ascending order;
12 keep the top-k values and their indexes as $\mathcal{R}.D'$, $\mathcal{R}.id$;
13 send $E_{pk_q}(\mathcal{R}.D')$ to QU;
14 (QU)
15 use sk_q to recover $E_{pk_q}(r_a \| r_b)$, $E_{pk_q}(\mathcal{R}.D')$;
16 **for** $v = 1$ *to* k **do**
17 compute $\bar{D}_v = \frac{\bar{D}'_v - r_b}{r_a}$;
18 return $\langle \mathcal{R}.D, \mathcal{R}.id \rangle$

Additionally, the identity information is obtained by executing $\mathbf{PRE.Dec}(sk_q, E_{rk_{dq}}(\mathcal{R}.ID))$. Moreover, QU acquires the exact top-k similarities values $\mathcal{R}.D$ by computing $\bar{D}_v = \frac{\bar{D}'_v - r_b}{r_a}$ for $v = 1, 2, \ldots, k$, which is shown in lines 14-18 of Algorithm 1.

C. Description of PTSRQ: A Privacy-Preserving Trajectory Similarity Range Query Scheme

This section describes PTSRQ, a privacy-preserving trajectory similarity range query scheme with a single cloud server. It comprises five stages, i.e., initialization, querying, filtration, refinement, and response, and the workflow is depicted in Fig. 5. Most dissimilar trajectories are also excluded through Hilbert-curve-based filtration, allowing the subsequent refined verification to be efficiently performed on a single cloud server to produce the final query results.

1) Initialization: For system initialization, DO generates a cluster of Hilbert curves $\mathcal{HC}$ and a set of keys for CKKS. Then, it establishes a data mapping relationship according to the Hilbert curve encoding and CKKS encryption. Next, it outsources the encrypted data to CS. Upon the registration of QU in the system, DO returns the encrypted curve parameters and generates a re-encryption key rk_{dq}. The details initialization process of PTSRQ is as follows:

Step 1. Data Pre-processing: DO first prepares the preprocessed trajectory data for the user queries, denoted as $\mathcal{T} = \{\langle ID_i, T^i\rangle\}_{i=1}^{n}$, where ID_i represents the moving object's identity, and $T^i = \{(t_1^i, p_1^i), (t_2^i, p_2^i), \ldots, (t_{l_i}^i, p_{l_i}^i)\}$ is the i-th trajectory in $\mathcal{T}$. The geo-location points on each trajectory are assumed to be located within the cells of a square grid with resolution of $2^N \times 2^N$, where the coordinates are integers.

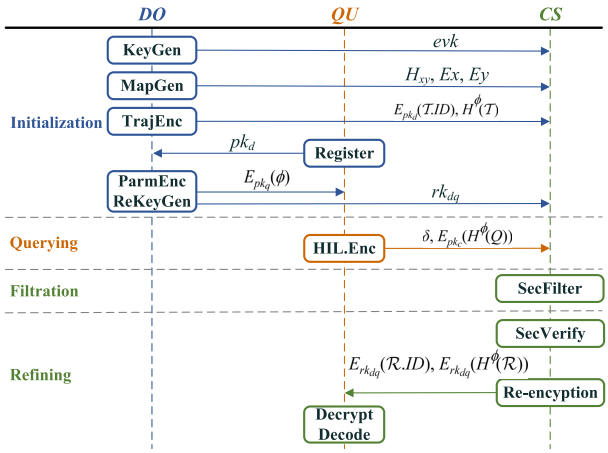


Fig. 5. System Flow Diagram for PTSRQ.

Step 2. Mapping Relationship Construction: DO runs $\mathbf{MapGen}$ to construct a Hilbert-based mapping relationship Λ, including a Hilbert-curve look-up table and a CKKS ciphertext array, which are obtained by following steps. First, DO selects a Hilbert curve parameter ϕ and generates a cluster of Hilbert curves through rotation and translation, denoted as $\mathcal{HC} = \{HC^{\phi}; HC^{I}, HC^{II}, \ldots, HC^{r}\}$, where r is the number of modified curves. For each coordinate point (x, y) in the target space, DO computes its Hilbert value H_{xy}^{η} under each curve HC^{η} using $\mathbf{HIL.Enc}((x, y), \phi^{\eta})$, where $x, y \in [0, 2^N - 1]$ and $\eta \in \{\phi, I, II, \ldots, r\}$. For CKKS encryption of the coordinate points corresponding to each H_{xy}^{η}, DO divides their x and y components into $d = \left\lceil \frac{4^N}{\ell} \right\rceil$ vectors each, with length ℓ. It then runs $\mathbf{CKKS.KeyGen}$ to generate a public key pk_d, a secret key sk_d, and an evaluation key evk. To leverage CKKS's SIMD capabilities, DO encodes those mentioned above $2d$ plaintext vectors into polynomial form and proceeds to encrypt them as

$$\begin{aligned} Ex &= \left\{E(x^{(1)}), \ldots, E(x^{(d)})\right\}, \\ Ey &= \left\{E(y^{(1)}), \ldots, E(y^{(d)})\right\}. \end{aligned} \tag{9}$$

Finally, DO establishes a mapping relationship Λ between the CKKS ciphertexts and each set of H-values H_{xy}^{η} according to the original coordinate plaintext. The map is denoted as Λ: $\mathcal{H}_{xy} \to [Ex, Ey]$, where the look-up table $\mathcal{H}_{xy} = \{H_{xy}^{\eta} | \eta \in \varphi\} = \langle H_{xy}^{\phi} : [H_{xy}^{I}, H_{xy}^{II}, \ldots, H_{xy}^{r}]\rangle$, $\varphi = \{\phi, I, II, \ldots, r\}$. The value H_{xy}^{ϕ} serves as the key in $\mathcal{H}_{xy}$ to assist subsequent data retrieval.

Step 3. Trajectory Encoding and Encryption: DO encodes the trajectories by executing $\mathbf{HIL.Enc}(p_j^i, \phi)$ to each coordinate point, resulting in $H^{\phi}(\mathcal{T}) = \{H^{\phi}(T^i) = [(t_1^i, H^{\phi}(p_1^i)), (t_2^i, H^{\phi}(p_2^i)), \ldots, (t_{l_i}^i, H^{\phi}(p_{l_i}^i))] | i = 1, 2, \ldots, n\}$. Additionally, DO executes $\mathbf{CKKS.Enc}(pk_d, ID)$ utilizing the

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 17,2026 at 03:39:16 UTC from IEEE Xplore. Restrictions apply.

public key to generate the encrypted trajectory identity information $E_{pk_d}(\mathcal{T}.ID) = \{E_{pk_d}(ID_i)|i = 1, 2, \ldots, n\}$.

Finally, DO outsources the lookup-table $\mathcal{H}_{xy}$, the encrypted array $[Ex, Ey]$, the encoded trajectory data $H^{\phi}(\mathcal{T})$ and the encrypted identity details $E_{pk_d}(\mathcal{T}.ID)$ to CS, along with the evaluation key evk.

Step 4. Query User Registration: To register for the similarity trajectory range query on DO's data, QU provides the public key pk_q for DO. DO then encrypts the selected Hilbert curve parameter ϕ using pk_q and executes $\mathbf{CKKS.ReKey}(sk_d, pk_d, pk_q)$ to generate a re-encryption key rk_{dq} for QU. At last, DO returns $E_{pk_q}(\phi)$ to QU and sends rk_{dq} to CS. Then, QU decrypts $E_{pk_q}(\phi)$ through sk_q to obtain the Hilbert curve parameters.

2) Querying: For a query trajectory $Q = [(t_1, q_1), (t_2, q_2), \ldots, (t_{l_0}, q_{l_0})]$, QU executes $\mathbf{HIL.Enc}(q_j, \phi)$ for each point in the query trajectory, obtaining the corresponding H^{ϕ}-values. This results in $H^{\phi}(Q) = [(t_1, H^{\phi}(q_1), (t_2, H^{\phi}(q_2), \ldots, (t_{l_0}, H^{\phi}(q_{l_0}))]$. Next, QU encrypts $H^{\phi}(Q)$ using CS's public key and sends the query request $\{E_{pk_c}(H^{\phi}(Q)), \delta\}$ to CS, where δ is the distance threshold.

3) Preliminary Filtration: In this phase, CS performs blind processing of similarity queries using the look-up table $\mathcal{H}_{xy}$ and the modified Hilbert-curve-based distance (refer to IV-A). Following this preliminary filtering, a subset of trajectories is selected as the candidate set $\mathcal{F}$.

Step 1. Hilbert-based Similarity Calculation: After receiving $H^{\phi}(Q)$, CS computes the Hilbert-based trajectory similarity $DSED_{\mathcal{H}}$ between the query trajectory Q and each trajectory $T^i \in \mathcal{T}$, i.e., $DSED_{\mathcal{H}}(Q, T^i) = (\frac{\Delta t_{1,2} \cdot s_1 + \Delta t_{h-1,h} \cdot s_h}{2} + \sum_{u=2}^{h-1} \frac{\Delta t_{u-1,u+1}}{2} \cdot s_u) \cdot \frac{1}{\Delta t_{1,h}}$, where the distance s_u between two spatial points is computed according to (7) with the assistance of the look-up table $\mathcal{H}_{xy}$ to calculate the modified Hilbert-curve-based distance. Consequently, similar to the description in Section IV-B3 of PTTSQ, CS obtains $D^{\mathcal{H}} = \{D_i^{\mathcal{H}} = DSED_{\mathcal{H}}(Q, T^i)|(i = 1, 2, \ldots, n)\}$.

Step 2. Trajectory Filtration: CS conducts a preliminary range screening on the trajectory data according to the order of trajectory similarity $\mathcal{D}^{\mathcal{H}}$, aiming to exclude the dissimilar ones and retain a portion of the trajectories as the candidate set for the range query result. The proportion of retained trajectories relative to the original whole trajectory dataset is set to $per_h \in (0, 1]$. For subsequent verification, CS records the indexes of these filtered candidate trajectories as $\mathcal{F}.id$.

4) Refinement: After the filtration, CS performs the homomorphic operations on the candidate trajectories to compute trajectory similarity, followed by secure range verification on their coordinate ciphertexts. Subsequently, it obtains the encrypted trajectory results that fall within the querying range along with their corresponding identities.

Step 1. Encrypted Trajectory Retrieval: For refined verification within the candidate trajectory set $\mathcal{F}$, CS first looks up the H-values of the coordinates in the query trajectory Q under the curve HC^{ϕ}. It then computes the coordinate ciphertext of Q through vector dot product operations with the ciphertext array $[Ex, Ey]$, denoted as $E(Q) = [(t_1, E(q_1)), (t_2, E(q_2)), \ldots, (t_{l_0}, E(q_{l_0}))]$. Similarly, based on the H-values of the coordinates in the candidate trajectories from $\mathcal{F}$, CS can obtain the corresponding ciphertexts of the candidate trajectories, i.e., $E(\mathcal{F}) = \{E(T_f^j) = [(t_1^j, E(p_1^j)), (t_2^j, E(p_2^j)), \ldots, (t_{l_j}^j, E(p_{l_j}^j))]|j \in \mathcal{F}.id\}$

Step 2. Homomorphic Calculation of Exact Distances: After obtaining the CKKS ciphertexts $E(\mathcal{F})$ and $E(Q)$, for each candidate trajectory T_f^j in the filtered trajectory result set $\mathcal{F}$, CS calculates the trajectory similarity DSED as defined by (3) in the ciphertext state. Specifically, by leveraging the homomorphic properties of CKKS, including homomorphic addition, multiplication, and inversion, the ciphertext of $DSED(Q, T_f^j)$, denoted as $E(D_j)$, can be computed as

$$E(D_j) = (\Delta t_{1,2} \cdot E(s_1^{(j)}) + \Delta t_{h_j-1,h_j} \cdot E(s_{h_j}^{(j)}) + \sum_{u=2}^{h_j-1} \Delta t_{u-1,u+1} \cdot E(s_u^{(j)})) \cdot \left(2 \cdot \Delta t_{1,h_j}\right)^{-1}, \tag{10}$$

where $h_j = l_0 + l_j - 2$, and l_0 and l_j are the trajectory lengths of Q and T_f^j, respectively, $j \in \mathcal{F}.id$, and $E(s_u^{(j)})$ is the encrypted distances between spatial points in the synchronous pairs obtained via homomorphic operations.

Step 3. Secure and Exact Range Verification: To extract the exact range query result trajectory set $\mathcal{R}$ from the preliminary filtered trajectory set $\mathcal{F}$, CS executes the secure range verification algorithm based on the secure comparison of CKKS ciphertexts, and obtains the trajectory ciphertexts that satisfy the distance threshold in range query.

To determine the relationship between the distance threshold δ and the similarity values D_j of each candidate trajectory $T_f^j \in \mathcal{F}$, CS homomorphically computes the sign result of their difference d_j in the ciphertext state, denoted as $E(\mu_j) = \text{sgn}(E(d_j))$. Next, the encrypted comparison result of the range query can be further obtained as

$$E(\sigma_j) = (E(\mu_j) + 1) \cdot 2^{-1} = (\text{sgn}(E(d_j)) + 1) \cdot 2^{-1},$$

where $E(\sigma_j)$ denotes the the ciphertext of the comparison result for T_f^j, and $E(d_j)$ denotes the ciphertext of the scaled difference between D_j and δ. In practice, the sign function $\text{sgn}(x)$ can be approximated by the composite polynomials [23], e.g., an approximation of $\text{sgn}(x)$ can be achieved via a polynomial f of degree $(2n + 1)$, i.e., $f_n(x) = \sum_{i=0}^{n} \frac{1}{4^i} \cdot \binom{2i}{i} \cdot x(1 - x^2)^i$.

If setting $n = 3$, the encrypted sign result $E(\mu_j)$ can be computed as $f_3(E(d_j))$, i.e., $E(\mu_j) = -\frac{5}{16} \cdot [E(d_j)]^7 + \frac{21}{16} \cdot [E(d_j)]^5 - \frac{35}{16} \cdot [E(d_i)]^3 + \frac{35}{16} \cdot E(d_j)$.

Thus, if the trajectory similarity DSED satisfies the query range, i.e., $D_j \leq \delta$, then the comparison result $E(\sigma_j) = E(1)$, otherwise it obtains $E(\sigma_j) = E(0)$, where $E(1)$ and $E(0)$ are the ciphertexts of approximate values corresponding to 1 and 0, respectively.

Upon obtaining the ciphertexts of comparison results $E(\sigma_j)$ as above, CS subsequently performs homomorphic multiplication to convert the trajectory encoding and identity identification

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 17,2026 at 03:39:16 UTC from IEEE Xplore. Restrictions apply.

information, respectively, as follows:

$$E_{pk_d}\left(H^{\phi}(\mathcal{R})\right) = \left\{E(\sigma_j)\cdot H^{\phi}(T_f^j)|j\in\mathcal{F}.id\right\},$$
$$E_{pk_d}\left(\mathcal{R}.ID\right) = \left\{E(\sigma_j)\cdot E(ID_f^j)|j\in\mathcal{F}.id\right\},$$

Thus, the ciphertexts of trajectories satisfying the query range condition are refined and represented as $E_{pk_d}(\mathcal{R}) = \langle E_{pk_d}(H^{\phi}(\mathcal{R})), E_{pk_d}(\mathcal{R}.ID)\rangle$.

5) Response: Finally, CS performs proxy re-encryption on the refined trajectory results, and their identities and returns these ciphertexts to QU in response to the range query. To retrieve plaintexts of query results, QU decrypts the trajectory results and identity information using its secret key.

Step 1. Proxy Re-encryption of Query Result: After refinement, for the proxy re-encryption of the verified result $E_{pk_d}(\mathcal{R})$, CS runs $\mathbf{CKKS.ReEnc}$ on both $E_{pk_d}(H^{\phi}(\mathcal{R}))$ and $E_{pk_d}(\mathcal{R}.ID)$ using the re-encryption key rk_{dq} of QU. Thus, the ciphertexts are transformed into $E_{rk_{dq}}(\mathcal{R}) = \langle E_{rk_{dq}}(H^{\phi}(\mathcal{R})), E_{rk_{dq}}(\mathcal{R}.ID)\rangle$, which is returned to QU as the query response.

Step 2. Query Result Recovery: After receiving $E_{rk_{dq}}(\mathcal{R})$ from CS, QU utilizes the secret key sk_q to retrieve the queried identity information $\mathcal{R}.ID$ by running $\mathbf{CKKS.Dec}(sk_q, E_{rk_{dq}}(\mathcal{R}.ID))$. To obtain the trajectory results of the range query, QU firstly runs $\mathbf{CKKS.Dec}(sk_q, E_{rk_{dq}}(H^{\phi}(\mathcal{R})))$, followed by decoding with $\mathbf{HIL.Dec}(\phi, H^{\phi}(\mathcal{R}))$. Ultimately, QU acquires the final query results $\mathcal{R}$.

V. Security Analysis

This section analyzes the security of PTTSQ and PTSRQ to verify the aforementioned security goals.

A. Security Analysis for PTTSQ

1) The Privacy of the DO's Data is Preserved in PTTSQ: During the system initialization phase of PTTSQ, the trajectory data $\mathcal{T}$ of DO is encoded via a Hilbert curve HC^{ϕ}, resulting in $H^{\phi}(\mathcal{T})$, which is then sent to CS_1. The curve parameter ϕ functions as a transformation key and is indispensable for decoding the encoded trajectories. Without knowledge of ϕ, both CS_1 and CS_2 cannot deduce DO's trajectories due to the blind evaluation mechanism based on Hilbert curve encoding [12]. Furthermore, although CS_1 can access to the encrypted trajectory data $E(\mathcal{F})$, it cannot recover the plaintext trajectories $\mathcal{T}$ without the secret key sk_d. Assuming that the two cloud servers do not collude, even if CS_2 holds sk_d, it can only retrieve the disturbed DSED similarity values D' from the encrypted similarities $E(D')$ using sk. The intermediate result reveal no information about the original trajectories in $\mathcal{T}$, so CS_2 is also incapable of revealing the trajectories. Thus, neither of the cloud servers can deduce any privacy details of the trajectories of DO. In the PTTSQ system, DO encrypts the identity information $\mathcal{T}.ID$ using pk_d before outsourcing, preventing CS_1 from deciphering it without sk_d. Even if CS_1 re-encrypts $E_{pk_d}(\mathcal{T}.ID)$ with rk_{dq}, the resulting ciphertexts $E_{rk_{dq}}(\mathcal{T}.ID)$ can only be decrypted with sk_q [17]. As neither cloud server possesses sk_d or sk_q, they are unable to infer the identity information associated with any trajectory. For a curious QU who does not collude with the two cloud servers, only the top-k trajectories $H^{\phi}(\mathcal{R})$ and their corresponding identities $E_{rk_{dq}}(\mathcal{R}.ID)$ are returned by the dual cloud servers after refinement. Even though QU can decrypt the trajectory results in $\mathcal{R}$ and their identities $\mathcal{R}.ID$ using ϕ and sk_q, respectively, it remains unable to deduce the rest of the trajectory data $\mathcal{T}-\mathcal{R}$. Therefore, the privacy of DO's trajectory data is preserved against all of CS_1, CS_2, and QU.

2) The Privacy of Query Request From the QU is Preserved in PTTSQ: In the PTTSQ system, the query request $E_{pk_c}(H^{\phi}(Q))$ from QU is generated by encoding the query trajectory Q using the Hilbert curve defined by DO, followed by encryption with the public key of CS_1. Although $H^{\phi}(Q)$ can be decrypted by CS_1 using the secret key sk_c, the actual query trajectory Q remains undisclosed to CS_1 without knowledge of the Hilbert curve's parameters chosen by DO. Moreover, DO and CS_2, which do not collude with CS_1, cannot access $H^{\phi}(Q)$ without CS_1's secret key sk_c, and therefore cannot retrieve QU's query request Q. As a result, the privacy of QU's query request is protected from DO, CS_1, and CS_2.

3) The Privacy of Query Results for the QU is Preserved in PTTSQ: After receiving the query responses $\{E_{rk_{dq}}(\mathcal{R}.ID), E_{pk_q}(H^{\phi}(\mathcal{R})), E_{pk_q}(r_a\|r_b)\}$ from CS_1 and the encrypted similarities $E_{pk_q}(\mathcal{R}.D')$ from CS_2, QU can use the secret key sk_q to obtain the query results $\mathcal{R} = \{\langle ID_j, R^j, D_j\rangle\}_{j=1}^{k}$. Without the secret key sk_q, the dual cloud servers CS_1 and CS_2 neither can disclose the identity information $\mathcal{R}.ID$ from $E_{rk_{dq}}(\mathcal{R}.ID)$, and nor deduce the trajectories from $H^{\phi}(\mathcal{R})$ due to the absence of the curve parameter ϕ. Furthermore, the requested DSED similarity values $\mathcal{R}.D$ are concealed from both cloud servers through the $\mathbf{SHE.SAC}$ protocol. In this process, CS_1 performs homomorphic operations based on SHE without access to the sk, while CS_2 recovers only the perturbed similarities D' without knowledge of r_a or r_b. By separately encrypting the two sets of ciphertexts $E_{pk_q}(D')$ and $E_{pk_q}(r_a\|r_b)$ before sending them back to QU, the true similarity values of the query results $\mathcal{R}.D$ are shielded from CS_1, CS_2, and DO. Despite DO possesses the trajectory dataset $\mathcal{T}$ and the Hilbert curve transformation key ϕ, it cannot deduce the queried top-k trajectories in $\mathcal{R}$ from $E_{pk_q}(H^{\phi}(\mathcal{R}))$ and $E_{pk_q}(\mathcal{R}.ID)$ due to the lack of sk_q. Moreover, the re-encryption ciphertext $E_{rk_{dq}}(\mathcal{R}.ID)$ is only recoverable using sk_q, rendering DO incapable of accessing the requested identities $\mathcal{R}.ID$. Consequently, the privacy of QU's query results, comprising the identity details, top-k trajectories, and their similarity values, are all preserved from CS_1, CS_2, and DO.

B. Security Analysis for PTSRQ

1) The Privacy of the DO's Data is Preserved in PTSRQ: Similarly, in PTSRQ, the trajectory data of DO is encoded using a Hilbert curve HC^{ϕ} before being uploaded to CS, denoted as $H^{\phi}(\mathcal{T})$. The parameter ϕ serves as a transformation key necessary for recovering the encoded trajectories [12]. As CS does not possess ϕ, it cannot infer DO's original trajectories. In addition, the identity information of the trajectories is encrypted as $E_{pk_d}(\mathcal{T}.ID)$, ensuring that $\mathcal{T}.ID$ remains undisclosed to

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 17,2026 at 03:39:16 UTC from IEEE Xplore. Restrictions apply.

CS without knowing the secret key sk_d. Although CS can re-encrypt $E_{pk_d}(\mathcal{T}.ID)$ using the re-encryption key rk_{dq}, the resulting ciphertexts $E_{rk_{dq}}(\mathcal{T}.ID)$ can only be decrypted with sk_q [17]. As CS holds neither sk_d nor sk_q, it is unable to derive the identity details of any trajectory. Considering a curious QU without collusion of CS, only the queried trajectories within the range threshold $H^{\phi}(\mathcal{R})$ and their identities $E_{rk_{dq}}(\mathcal{R}.ID)$ are returned by the CS. Even though QU can recover the queried trajectories in $\mathcal{R}$ and their identity information $\mathcal{R}.ID$ using ϕ and sk_q accordingly, it cannot deduce the remaining trajectory data $\mathcal{T}-\mathcal{R}$. Therefore, the privacy of DO's trajectory data is protected against both CS and QU.

2) The Privacy of Query Request From the QU is Preserved in PTSRQ: In our PTSRQ, the query request $E_{pk_c}(H^{\phi}(Q))$ from QU is generated by encoding the query trajectory Q using the Hilbert curve defined by DO, and then encrypting it by using the public key of CS. Although $H^{\phi}(Q)$ can be decrypted by CS using the secret key sk_c, the plaintext of the query trajectory Q remains undisclosed to CS without knowledge of the Hilbert curve's parameters held by DO. In addition, under the non-collusion assumption of DO and CS, DO cannot obtain $H^{\phi}(Q)$ without CS's secret key sk_c, and thus cannot get QU's query request Q. Consequently, the privacy of QU's query request is preserved from both DO and CS.

3) The Privacy of Query Results for the QU is Preserved in PTSRQ: After receiving the query responses $E_{rk_{dq}}(H^{\phi}(\mathcal{R}))$ and $E_{rk_{dq}}(\mathcal{R}.ID)$ from CS, QU can use the secret key sk_q to obtain the query results $\mathcal{R}=\{\langle ID_j, R^j\rangle\}_{j=1}^{k}$. Since the re-encrypted ciphertexts $E_{rk_{dq}}(\mathcal{R}.ID)$ can only be decrypted with sk_q, CS, lacking this key, is unable to reveal either the encoded trajectories $H^{\phi}(\mathcal{R})$ or their identity information $\mathcal{R}.ID$ from the encrypted query results $E_{rk_{dq}}(\mathcal{R})$. Likewise, although DO possesses the complete trajectory data $\mathcal{T}$ and the Hilbert curve transformation key ϕ, it cannot deduce the queried trajectories in $\mathcal{R}$ from $E_{rk_{dq}}(H^{\phi}(\mathcal{R}))$ due to the absence of sk_q. Therefore, the privacy of the queried trajectories and their corresponding identity details are preserved from both CS and DO.

VI. Performance Evaluation

This section first states our experimental settings and then evaluates our PTTSQ and PTSRQ in terms of filtration precision, communication overheads and computational cost.

A. Experimental Settings

To evaluate the performance of our schemes, we conduct experiments on a Dell Precision 7920 Tower Server with two Intel Xeon Gold 6248R (96) @4 GHz CPU and 128 GB RAM running Ubuntu 20.04 using Python 3.8.

Four trajectory datasets are used for evaluation, including two real-world trajectory datasets known as *T-drive* [24] and *Geolife* [25], as well as two synthetic datasets generated via a random walk algorithm with a *uniform* and a *normal* pace distribution. The numbers of trajectories n in *T-drive*, *Geolife*, *Uniform* and *Normal* are $2000, 2000, 5000$ and 5000, respectively, with corresponding trajectory lengths l of 100, 200, 100 and 200.

During the evaluation, the querying map is configured with a grid size of $2^N \times 2^N$ for $N \in \{8, 9, 10, 11\}$, with the querying resolution ranging from 65,536 up to 4,194,304. For initial filtration, a cluster of six Hilbert curves $\mathcal{HC} = \{HC^{\phi}; HC^{I}, HC^{II}, \ldots, HC^{VI}\}$ is used for the modified Hilbert-curve-based distance calculation of trajectory similarity, where $\mathcal{HC}$ is created from the original Hilbert curve parameter $\phi = \{11, \sqcap, (0,0), 5\,\mathrm{m}\}$ through rotations with $\theta \in \{\sqcup, \sqsubset, \sqsupset\}$ and shifting the starting point to $P_0 \in \{(1,1), (2,2), (4,4)\}$. Due to the recursive and self-similar structure of the Hilbert curve, its locality-preserving properties are proven to remain invariant under rotation and translation transformations [19], [20]. This invariance ensures that the performance of the proposed filtration system, which leverages these locality-preserving properties, is unaffected by the orientation θ or starting point P_0 of the Hilbert curve. Therefore, our experimental evaluation focuses on the generation of curve clusters based on the predefined HC^{ϕ} framework as described above, omitting detailed parametric analysis of the original curve itself. For comparison, we set a baseline scheme that calculates the similarity using only a single curve HC^{ϕ}. In the SHE encryption system, the security parameters k_0, k_1 and k_2 are set to 2048, 24, and 160, respectively. For the CKKS encryption system, the security parameters $\{Polynomial\ Modulus,\ Coefficient\ Modulus,\ Scale\}$ are set to $\{8192, [60, 40, 40, 60], 2^{40}\}$. In addition, given a user's query trajectory Q, the top-k query values are set to $k = 5, 10, 15, 20$, and the distance threshold in the trajectory range queries are set to $per_0 = \{0.001, 0.002, 0.003, 0.004\}$-quantile. The implementation code and preprocessed datasets are available at: https://github.com/Sop-ykk/PTTSQ-PTSRQ/.

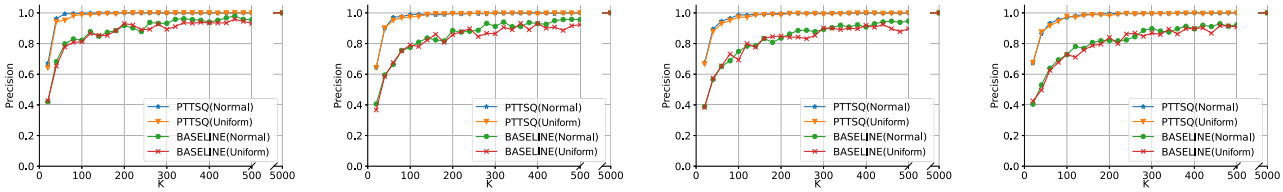
B. Evaluations and Experimental Results of PTTSQ

We first evaluate the filtration precision of our privacy-preserving top-k trajectory similarity query scheme (PTTSQ), followed by an analysis of communication overhead. Finally, we compare the system initialization and query processing time on both real-world and synthetic datasets with recent studies [7], [8].

1) The Filtration Precision: To address the issue of low similarities between adjacent points in a single Hilbert curve, the modified Hilbert-curve-based distance is introduced, employing a cluster of Hilbert curves to enhance query precision. To evaluate the effectiveness of this modified filtration scheme, experiments are conducted comparing filtration with and without the modification. The filtration precision is defined as the ratio of the cardinality of the intersection between the preliminarily filtered result set R and the actual top-k similar trajectory set C to the size of C, i.e., $pr = \frac{|R \cap C|}{|C|}$. For the ideal case, where $\mathcal{C} \subseteq \mathcal{R}$, the precision pr reaches 100%.

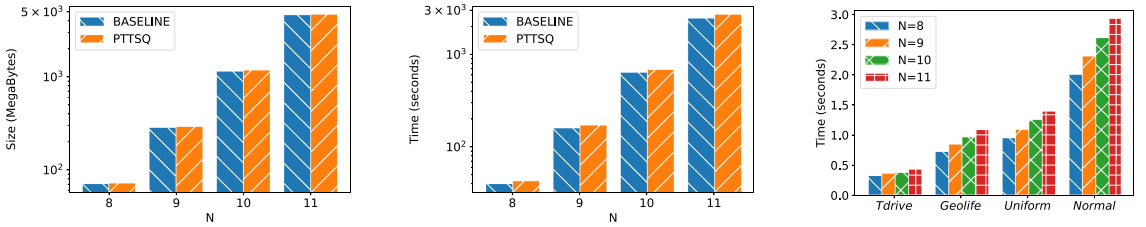
We conduct top-k similarity queries on the *Uniform* and *Normal* datasets, each comprising 5,000 trajectories, by varying the filtration scope from k to n for top-5, top-10, top-15, and top-20 queries. The experiments are repeated 100 times to calculate the average precision results for each scenario. The filtration precision comparison results are illustrated in Fig. 6, where the modified Hilbert distance-based PTTSQ, utilizing a cluster of

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 17,2026 at 03:39:16 UTC from IEEE Xplore. Restrictions apply.



(a) The filtration precision ($k = 5$). (b) The filtration precision ($k = 10$). (c) The filtration precision ($k = 15$). (d) The filtration precision ($k = 20$).

Fig. 6. The filtration precision of PTTSQ varying with K for $k = 5, 10, 15, 20$.



(a) The size of Γ. (b) The time of constructing Γ. (c) The time of Hilbert curve encoding.

Fig. 7. The performance of Γ construction and trajectory encoding during the initialization phase.

curves $\mathcal{HC}$, is compared to the baseline that utilizes only a single curve HC^{ϕ}. As illustrated, PTTSQ significantly enhances filtration precision over the baseline for both *Uniform* and *Normal* datasets. Specifically, for the tested top-k similarity queries, our filtration technique achieves over 99% precision when the filtration range is set to $K = 125, 190, 215, 220$, respectively. These results demonstrate that PTTSQ's initial filtering mechanism effectively reduces computational overhead while preserving high query accuracy.

2) Communication Overheads: For simplicity, we denote the total number of grids in the target map as $\Omega = 4^N$. The operations of SHE encryption, Hilbert curve encoding, public key encryption, PRE encryption, PRE re-encryption are denoted as $\{E_S, E_H, E_P, E_{R_E}, E_{R_R}\}$, with their respective ciphertext lengths represented as $\{|E_S|, |E_H|, |E_P|, |E_{R_E}|, |E_{R_R}|\}$. Similarly, the corresponding decryption or decoding operations are represented by $\{D_S, D_H, D_P, D_R\}$.

During the initialization phase of PTTSQ, the primary communication overhead arises from transmitting the Hilbert-based look-up table Γ, which requires $\Omega \cdot [(1 + r) \cdot |E_H| + 2 \cdot |E_S|]$ bits. The size of the look-up table Γ with different Hilbert curve order N is shown in Fig. 7(a), where our modified Hilbert-curve-based PTTSQ scheme is compared to the baseline scheme without modification. As illustrated, incorporating modifications into the filtration mechanism incurs minimal additional communication overhead, while significantly improving query precision, as shown in Fig. 6. Furthermore, the communication overhead for outsourcing the trajectory data and the query request is $\sum_{i=1}^{n} l_i \cdot |E_H| + n \cdot |E_{R_E}| + (1 + l_0) \cdot |E_P|$ bits. Then, the top-k query is primarily processed through Hilbert curves-based filtering, taking $K \cdot |E_S|$ bits of communication for CS_1 to transmit the initial filtration results to CS_2. During the refinement phase, CS_1 transmits the encrypted random numbers and the encrypted top-k results to QU, with communication overheads of $|E_P|$ bits and $k \cdot |E_{R_R}| + \sum_{j=1}^{k} l_j \cdot |E_P|$ bits, respectively. Moreover, CS_2 sends the ciphertexts of the similarity values to QU, requiring $k \cdot |E_P|$ bits of communication.

3) Computational Cost: During the initialization phase, DO runs **TableGen** to construct Γ, which requires $(1 + r)\Omega$ executions of E_H to produce Ω key-value pairs in the Hilbert-curve-based look-up table, and 2Ω executions of E_S to encrypt the corresponding coordinates. Subsequently, for the data outsourcing and query launching, encrypting the trajectory data requires $(l_0 + \sum_{i=1}^{n} l_i)$ executions of E_H, n executions of E_{R_E}, one execution of E_P, and one call to **PRE.ReKey**.

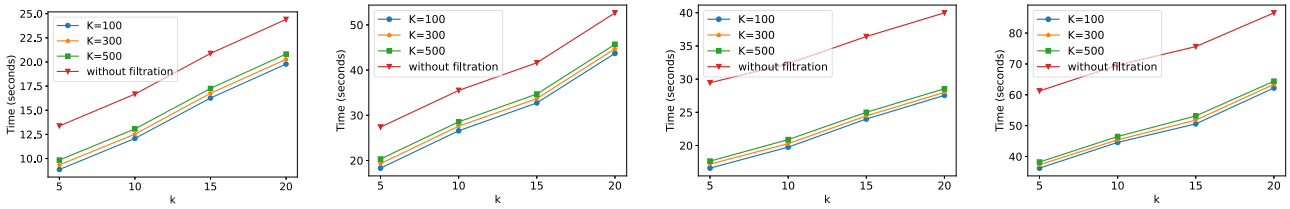
During query processing phase, CS_1 performs initial trajectory filtering and collaborates with CS_2 for result refinement, which involves one **Hil.Filt** and one **SHE.SAC**, comprising $\mathcal{O}(nl)$ homomorphic operations and K executions of D_S. Additionally, generating encrypted query results requires $(1 + k + \sum_{j=1}^{k} l_j)$ executions of E_P and k executions of E_{R_R}.

Finally, QU performs k executions of D_{R_R}, and $\sum_{j=1}^{k} l_j$ executions each of D_P and D_H to obtain the query results.

To better demonstrate the computational performance of our PTTSQ scheme, we conduct extensive experiments, with the results presented below.

The Time of the Table Construction: To exhibit the time required for table construction, we generate look-up tables using a cluster of 6 Hilbert curves $\mathcal{HC}$ (as described in Section VI-A) for each curve order $N \in \{8, 9, 10, 11\}$, and then compare the construction time with the baseline method. As shown in Fig. 7(b), the construction time of Γ is primarily influenced by the order N of the chosen curve, with modifications having only a minor impact. This is because, as N increases, more information becomes available due to the total of 2^{2N} points in the space. Moreover, there is no notable distinction between adding modifications and expanding the cluster size of the

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 17,2026 at 03:39:16 UTC from IEEE Xplore. Restrictions apply.



(a) *Tdrive*. (b) *Geolife*. (c) *Uniform*. (d) *Normal*.

Fig. 8. The query processing time of PTTSQ over different datasets.

TABLE II
TOTAL TIME COST OF TOP-k QUERY SCHEMES ON FOUR DATASETS WITH 50 TRAJECTORIES AND $N = 8, 9, 10, 11$

	System Initialization (sec)	**Query Processing** (sec)	**Ref. [7]** (sec)	**Ref. [8]** (sec)
	Order 8 / 9 / 10 / 11	Order 8 / 9 / 10 / 11	Order 8 / 9 / 10 / 11	Order 8 / 9 / 10 / 11
T-drive	43.0 / 173.6 / 698.6 / 2753.1	0.4184 / 0.4155 / 0.4156 / 0.4145	3863.5 / 4759.4 / 8167.7 / 10456.1	6201.3 / 6206.1 / 6674.2 / 23806.5
Geolife	43.0 / 173.6 / 698.6 / 2753.1	0.4166 / 0.4137 / 0.4146 / 0.4158	1303.8 / 1888.2 / 3122.5 / 4236.4	5959.8 / 6029.1 / 5969.6 / 7427.5
Uniform	43.0 / 173.6 / 698.6 / 2753.1	0.4180 / 0.4162 / 0.4150 / 0.4157	2130.1 / 4532.5 / 9506.9 / 13795.9	6176.8 / 6242.4 / 9340.1 / 21780.6
Normal	43.0 / 173.6 / 698.6 / 2753.1	0.4196 / 0.4165 / 0.4177 / 0.4166	4661.1 / 7363.5 / 12513.6 / 16459.8	5966.5 / 5969.5 / 11331.3 / 20849.4

curves. Therefore, it is practical to establish a look-up table incorporating various transformations to adjust the approximation of trajectory similarities.

The Time of Trajectory Encoding: The time cost of encoding trajectories using the Hilbert curve is evaluated across four datasets: *T-drive*, *Geolife*, *Uniform*, and *Normal*. As shown in Fig. 7(c), the encoding time increases with the dataset size, measured by the number of trajectories n and their average length l, as well as the curve order. Nevertheless, the process remains highly efficient, with time consumption at the second level.

The Time of Filtration and Refining: We conduct experiments using a look-up table of 11th-order Hilbert curve as described earlier. To demonstrate the efficiency of our PTTSQ method, we run simulations for top-5, top-10, top-15, and top-20 similarity queries on four datasets. By varying the filtering scope $K = 100, 300, 500$, we generate the top-k query results to meet different precision requirements. The query processing time for our filter-and-refine approach in PTTSQ is illustrated in Fig. 8, which includes the initial filtration time (t_F), refined search time (t_R), and query result recovery time. Since t_F involves filtering a large number of trajectories at high speed, while t_R focuses on refined searching within a smaller subset of trajectories ($K \ll n$), the filtration step for large-scale trajectory datasets proves to be highly efficient.

Total Time Cost: In this experiment, we evaluate the overall query performance of our PTTSQ by comparing it with two related schemes, STFSM [7] and PDRQ [8], through simulations of top-k queries on four datasets. Due to the low query efficiency of the comparison schemes, we randomly select 50 trajectories of length 50 from each dataset for the experiments and perform top-10 queries on them respectively. The comparison results of total query time are displayed in Table II. According to the table, for all four datasets and varying curve order N, both STFSM [7] and PDRQ [8] take thousands or tens of thousands of seconds to execute a query. In contrast, our PTTSQ approach takes tens or thousands of seconds to construct the look-up table Γ, while completing the online query in just one second.

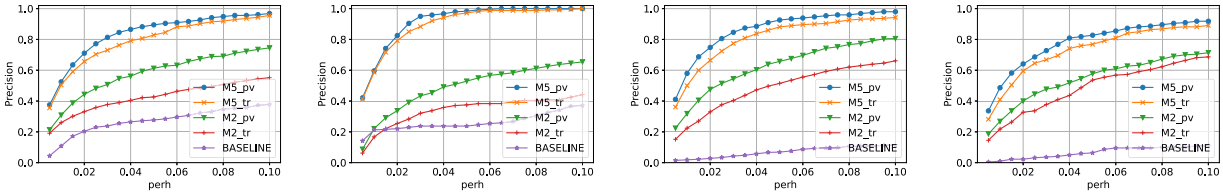
The experimental results show that STFSM [7] and PDRQ [8] are time-consuming even when dealing with a dataset of only 50 trajectories, making them unsuitable for larger datasets. In contrast, as shown in Fig. 8, PTTSQ requires less than 70 seconds for a top-20 query among 5000 trajectories, significantly outperforming STFSM and PDRQ on just 50 trajectories. Moreover, the look-up table Γ in PTTSQ can be pre-constructed during system initialization and reused across different queries. As further demonstrated in Fig. 6, PTTSQ achieves high precision in top-k queries, and is effective for large-scale trajectory datasets.

C. Evaluations and Experimental Results of PTSRQ

We first evaluate our privacy-preserving trajectory similarity range query scheme (PTSRQ) from the following three aspects, i.e., the filtration precision, the communication overheads, and the computational costs. Then, we compare the query time with other related works [7], [8].

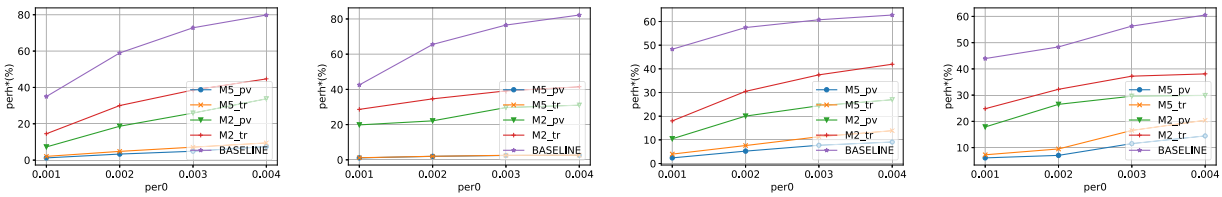
1) The Filtration Precision: To overcome the limitations of single Hilbert curve-based filtering, we propose enhanced filtration methods that integrate multiple Hilbert curves and pivot points, thereby improving range query precision. To evaluate filtration precision, we implement PTSRQ on four different datasets, setting the query range threshold to $per_0 = 0.004$. Each experiment is repeated 100 times in each case, and average precision results are reported. Filtration precision (pr) of the range query is defined as $pr = \frac{|\mathcal{R} \cap \mathcal{C}|}{|\mathcal{C}|}$, where $\mathcal{R}$ represents the set of returned trajectories through range query and $\mathcal{C}$ represents the actual set of trajectories satisfying the range threshold. Meanwhile, the minimum filtering ratio (per_h^*) to achieve an accurate query is also recorded in each query scenario. Once the filtering ratio per_h meets or exceeds per_h^*, the PTSRQ scheme achieves full accuracy with $pr = 100\%$.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 17,2026 at 03:39:16 UTC from IEEE Xplore. Restrictions apply.



(a) *T-drive* (b) *Geolife* (c) *Uniform* (d) *Normal*

Fig. 9. The filtration precision of PTSRQ on different datasets varying with per_h.



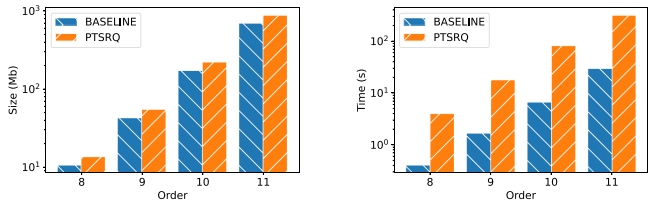
(a) *T-drive* (b) *Geolife* (c) *Uniform* (d) *Normal*

Fig. 10. The minimum filtering ratio per_h^* for accurate queries for $per_0 = 0.001, 0.002, 0.003, 0.004$.

The precision results of PTSRQ under different modification settings are shown in Fig. 9, where M<r>_pv represents the HD_p modified by pivot-points, M<r>_tr represents the HD_t modified by transformations, and r denotes the number of Hilbert curves in the cluster $\mathcal{HC}$. The precision results of PTSRQ are compared with the baseline approach ($r = 0$) for initial filtration with only HD based on the original curve HC^{ϕ}. As can be seen from Fig. 9, when the filtering ratio reaches $per_h = 0.05$, the average query precision of our PTSRQ scheme closes to 90%. Compared to the HD_t-based filtering, the HD_p-based filtering method achieves higher filtering precision pr and requires a smaller minimum filtering ratio per_h^*. Furthermore, increasing the number of curves r in the Hilbert curve enhances the performance of the Hilbert-curve-based distance approximation in terms of precision. The minimum filtering ratio (per_h^*) required for accurate range queries on different trajectory datasets is shown in Fig. 10, further confirming that the modified Hilbert distance based PTSRQ scheme, including pivot-point modification (HD_p) and transformation modification (HD_t), achieves higher query precision than the baseline scheme without modification. Moreover, under the same cluster of Hilbert curves, the filtering scheme based on HD_p performs better than HD_t, highlighting the effectiveness of pivot-point modification in improving range query accuracy.

2) Communication Overheads: For simplicity, we also refer to Ω as the total number of grids in the target map, which is 4^N. The operations of Hilbert curve encoding, public key encryption, CKKS encryption and CKKS re-encryption algorithms are denoted as $\{E_H, E_P, E_{C_E}, E_{C_R}\}$, respectively, with their ciphertext lengths expressed as $\{|E_H|, |E_P|, |E_C|\}$. Similarly, the corresponding decryption or decoding operations are represented as $\{D_H, D_P, D_C\}$.

In the system initialization phase, DO sends the Hilbert-based mapping relationship $\Lambda : \mathcal{H}_{xy} \rightarrow [Ex, Ey]$ to CS. This include the look-up table $\mathcal{H}_{xy} = \{H_{xy}^{\eta} | \eta \in \phi\}$ and two sets of CKKS ciphertext vectors, incurring a communication overhead of $\Omega \cdot (1 + r) \cdot |E_H| + 2 \cdot \lceil \frac{\Omega}{\ell} \rceil \cdot |E_C|$ bits. The size of the mapping relationship Λ under different Hilbert curve orders N is displayed in Fig. 11(a), where our PTSRQ with modified Hilbert-curve-based trajectory filtration method is compared with the baseline schemes without modification. As observed in the figure, these modifications introduce only negligible additional communication overhead. Additionally, the communication overhead of outsourcing trajectory data and query requests is $\sum_{i=1}^{n} l_i \cdot |E_H| + n \cdot |E_C| + (1 + l_0) \cdot |E_P|$ bits.



(a) The size of Λ. (b) The time of constructing Λ

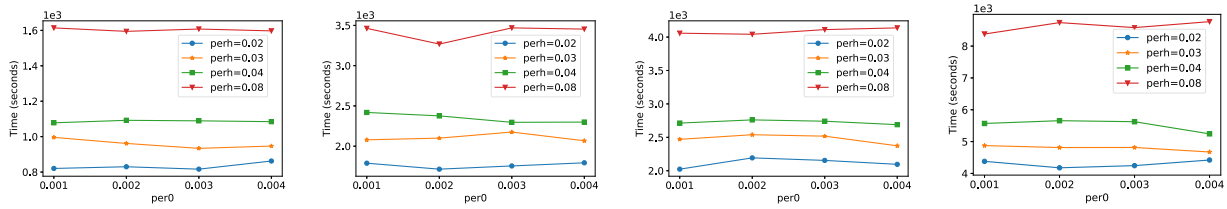
Fig. 11. The size and construction time of Λ.

The query is executed on the cloud server using a filter-and-refine strategy. In PTSRQ, the trajectory similarity range query is processed solely by a single cloud server CS, eliminating intermediate communications during the filtration and refinement phases. To return the final query results of PTSRQ, CS sends the encrypted trajectory data along with their identities to QU, incurring a communication overhead of $(\sum_{i=1}^{|\mathcal{F}|} l_i + |\mathcal{F}|) \cdot |E_C|$, where $|\mathcal{F}|$ denotes the number of trajectories in the candidate set after initial filtration.

3) Computational Cost: In the system initialization phase, DO executes **MapGen** to construct Λ, which takes $(1 + r)\Omega$ executions of E_H to generate Ω key-value pairs for the Hilbert-curve-based look-up table, and $2 \cdot \lceil \frac{\Omega}{\ell} \rceil$ executions of E_C to encrypt the coordinates. Then, the encryption of trajectory data for data outsourcing and querying consumes $(l_0 + \sum_{i=1}^{n} l_i)$ executions of E_H, n executions of E_C, one executions of E_P and one call of **CKKS.ReKey**.

In the query processing of PTSRQ, a single cloud CS is responsible for executing the Hilbert-curve-based trajectory filtration algorithm **Hil.Filt**, followed by the security verification

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 17,2026 at 03:39:16 UTC from IEEE Xplore. Restrictions apply.



(a) *Tdrive* (b) *Geolife* (c) *Uniform* (d) *Normal*

Fig. 12. The query processing time of PTSRQ over different datasets.

algorithm **CKKS.Verify** to refine the candidate trajectory results, involving $\mathcal{O}(nl)$ homomorphic operations. In addition, it takes $(|\mathcal{F}| + \sum_{j=1}^{|\mathcal{F}|} l_j)$ executions of E_C to generate the encrypted query results. Finally, QU performs $(|\mathcal{F}| + \sum_{j=1}^{|\mathcal{F}|} l_j)$ executions of D_R and $\sum_{j=1}^{|\mathcal{F}|} l_j$ executions of D_H to recover the final query result.

To better illustrate the computational efficiency, we implement our PTSRQ and show the performance results as follows.

The Time of the Map Construction: To evaluate the computational costs of constructing the mapping relationship $\Lambda : \mathcal{H}_{xy} \rightarrow [Ex, Ey]$, we build the look-up table $\mathcal{H}_{xy}$ based on a cluster of Hilbert curves $\mathcal{HC}$ (as in Section VI-A) for each curve order $N \in \{8, 9, 10, 11\}$, and obtain the corresponding CKKS ciphertext vectors Ex and Ey. The construction time of Λ in PTSRQ is compared with that of the baseline scheme, as displayed in Fig. 11(b). It can be observed that the construction time of Λ is mainly influenced by the curve order N, and the introduced modifications have only a slight impact. Furthermore, in comparison with the construction time of the look-up table in the PTTSQ scheme (illustrated in Fig. 7(a)), our PTSRQ reduces the computational costs by utilizing the SIMD capabilities of CKKS, thereby enhancing the efficiency of system initialization.

The Time of Filtration and Refining: Based on a cluster of 11-order Hilbert curves generated from $\phi = \{11, \sqcap, (0, 0), 1\}$, the trajectories in our PTSRQ scheme are first filtered and then refined. We perform the scheme for trajectory range queries on each dataset with different threshold requests, where the proportion of trajectories satisfying the query threshold is set to $per_0 = 0.001, 0.002, 0.003, 0.004$. The query processing times of our PTSRQ scheme are shown in Fig. 12, where the filtering ratio $per_h = 0.02, 0.03, 0.04, 0.08$ indicating the proportion of candidate trajectories retained after filtration. A larger per_h may achieve higher precision but incurs greater computational cost. Notably, setting $per_h = 100\%$ implies direct refinement over all n trajectories without initial filtration. The figure shows that the query processing time increases with per_h. Since t_F, the filtration time over n trajectories, is highly efficient, and t_R, the refinement time, is performed on a relatively small subset, the filter-and-refine strategy proves particularly effective for large-scale trajectory datasets.

Total Time Cost: The overall query efficiency of our PTSRQ is evaluated by comparison with STFSM [7] and PDRQ [8], using simulations of range similarity queries across four datasets. In the comparative experiments, 50 trajectories of length 50 are randomly selected from each dataset, with the query threshold in PTSRQ set to $per_0 = 0.1$. The total query processing times are shown in Table III. As shown, for all datasets and different order N, both STFSM [7] and PDRQ [8] require thousands or tens of thousands of seconds to perform a query. In contrast, our PTSRQ scheme constructs the mapping relationship Λ within hundreds of seconds, achieving nearly an order-of-magnitude reduction in system initialization time compared to PTTSQ. Moreover, the range query processing on a single cloud server in PTSRQ completes within 100 seconds, demonstrating significant improvement in overall efficiency.

The experimental results indicate that the query processing time of our PTSRQ scheme over 2000 or 5000 trajectories is lower than the query time of STFSM [7] and PDRQ [8] operating on 50 trajectories. Notably, the mapping relationship Λ can be precomputed during the system initialization phase and requires construction only once for different queries. Moreover, as depicted in Fig. 9, our PTSRQ can also achieve high-precision range queries. These results demonstrate PTSRQ's effectiveness for large-scale trajectory datasets.

D. Comparative Analysis of PTTSQ and PTSRQ

To offer practical guidance for application, we give a comparative analysis of the dual-cloud-based PTTSQ scheme and the single-cloud-based PTSRQ scheme.

1) Security Trade-Offs: The dual-cloud model adopted by PTTSQ offers stronger resilience against single-cloud compromise by distributing trust. While PTTSQ ensures trajectory privacy through specially designed secure protocols, its security weakens under potential collusion between cloud providers. In such scenarios, the single-cloud-based PTSRQ scheme becomes a more suitable choice, as it avoids the risks associated with inter-provider trust assumptions.

2) Communication Overhead: As illustrated by the system flow diagrams in Figs. 4 and 5, the single-cloud-based PTSRQ scheme incurs lower communication costs, primarily due to the elimination of one round of interaction between two cloud servers required by the SHE.SAC protocol in the PTTSQ scheme. Furthermore, as shown in Figs. 7(a) and 11(a), the Hilbert-curve-based mapping relationship Λ in PTSRQ achieves an 80% reduction in storage size compared to the look-up table Γ used in PTTSQ.

3) Computational Costs: The experimental results highlight distinct computational efficiency advantages of each scheme across different phases, as displayed in Table II and Table III. Specifically, the single-cloud-based PTSRQ exhibits superior efficiency during the system initialization phase, achieving an 85-90% reduction in computation time compared to its

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 17,2026 at 03:39:16 UTC from IEEE Xplore. Restrictions apply.

TABLE III
TOTAL TIME COST OF RANGE QUERY SCHEMES ON FOUR DATASETS WITH 50 TRAJECTORIES AND $N = 8, 9, 10, 11$

	System Initialization (sec)	**Query Processing** (sec)	**Ref. [7]** (sec)	**Ref. [8]** (sec)
	Order 8 / 9 / 10 / 11	Order 8 / 9 / 10 / 11	Order 8 / 9 / 10 / 11	Order 8 / 9 / 10 / 11
T-drive	4.23 / 17.98 / 80.61 / 314.8	92.25 / 94.36 / 93.41 / 91.42	3863.5 / 4759.4 / 8167.7 / 10456.1	6201.3 / 6206.1 / 6674.2 / 23806.5
Geolife	4.23 / 17.98 / 80.61 / 314.8	94.18 / 94.31 / 94.94 / 93.02	1303.8 / 1888.2 / 3122.5 / 4236.4	5959.8 / 6029.1 / 5969.6 / 7427.5
Uniform	4.23 / 17.98 / 80.61 / 314.8	93.14 / 93.77 / 92.01 / 91.62	2130.1 / 4532.5 / 9506.9 / 13795.9	6176.8 / 6242.4 / 9340.1 / 21780.6
Normal	4.23 / 17.98 / 80.61 / 314.8	92.42 / 94.60 / 95.31 / 91.53	4661.1 / 7363.5 / 12513.6 / 16459.8	5966.5 / 5969.5 / 11331.3 / 20849.4

dual-cloud-based PTTSQ. This efficiency gain is attributed to the improved parallel computation framework, which leverages the SIMD capabilities of CKKS encryption to vectorize the mapping relationship. In contrast, during the refined query processing phase, the dual-cloud-based PTTSQ demonstrates significantly better performance: PTTSQ completes refined queries within seconds, whereas PTSRQ requires minutes, and other related approaches may take over an hour. These phase-specific efficiency characteristics indicate that practical system design should account for the anticipated workload balance between initialization and query execution. For high-concurrency environments, PTTSQ offers better latency performance for lightweight queries. Conversely, PTSRQ provides faster initialization for range queries and substantially lower computational and communication overhead throughout the process, making it well-suited for bandwidth-limited scenarios or applications requiring frequent system reconfigurations.

VII. RELATED WORKS

A. Similarity Query for Trajectory Data

Research on trajectory similarity queries has evolved from the traditional kNN query, which aims to identify k nearest neighboring points to a given query point. With the development of mobile devices, the kNN queries have gradually shifted from static points to the trajectories of moving objects. To enable efficient similarity retrieval of large-scale trajectory data, various data structures and spatial indexing techniques have been explored in the literature. Guibas et al. [26] proposed an algorithm for maintaining the Voronoi diagram over time, offering numerical stability in updating the topological structure to support moving objects queries. Frentzos et al. [27] employed an R-tree-like index within a spatio-temporal database to facilitate trajectory similarity queries. Liang et al. [28] introduced grid-based indexing and query partitioning techniques over trajectory databases to search top-k similar trajectories.

To measure the similarity between trajectories, various calculation methods have been proposed, including dynamic time warping (DTW) [29], longest common subsequence (LCSS) [30], edit distance on real sequence (EDR) [31], etc. These approaches rely on dynamic programming to align sample points, providing effective measures for trajectory similarity. Existing studies have made substantial progress in handling noise interference and sampling rate variation, enhancing both the accuracy and efficiency of similarity queries in plaintext environments. However, when privacy preservation is taken into account, significant challenges remain, warranting further exploration.

B. Privacy-Preserving Methods for Trajectory Data

Existing privacy-preserving schemes for trajectory data can be categorized into three types, anonymization-based, differential-privacy-based, and encryption-based schemes.

Anonymization-based methods [2], [3] preserve trajectory data from being distinguished mainly by generating the quasi-identifiers in the dataset. Commonly employed techniques include k-anonymity, l-diversity, and t-closeness.

Differential-privacy-based methods [4], [5] typically introduce noise to the output, thus protecting individual trajectory information from inference after publication.

These two types of methods offer solutions for trajectory data privacy protection, but remain vulnerable to attacks leveraging background knowledge and may not be able to ensure the security of trajectory data sufficiently. In addition, they may lose some positional accuracy while achieving privacy protection, and impact the query precision. As a result, they may not fully satisfy the requirements of privacy protection trajectory similarity queries.

Towards this end, encryption-based methods [6], [7], [8] conceal private information from adversaries using cryptographic techniques, offering stronger security guarantees at the cost of increased computational cost. Existing research on privacy-preserving spatial queries has primarily focused on range and kNN queries over encrypted point data, with recent advancements extending to top-k spatial keyword queries [32], [33], [34], [35]. These schemes typically integrate encryption algorithms with various indexing structures to achieve efficient query processing. However, most current approaches are designed for static spatial data and cannot be directly applied to trajectory scenarios due to the inherent dynamics of trajectory data. Although some preliminary progress has been achieved, developing optimized solutions for dynamic trajectory data remains an open challenge. To this end, Liu et al. [6] first studied the secure similarity computation on encrypted trajectories, but their work focuses solely on pairwise similarity computations between encrypted trajectories, which leads to substantial computational complexity when querying large datasets. To improve the search efficiency, Teng et al. [7] proposed a Secure Trajectory Filtering based on Signature Matching (STFSM) method. However, this approach still incurs high computational costs due to the need to determine point-to-line segment positional relationships in the ciphertext state. Guan et al. [8] proposed a privacy-preserving scheme for trajectory range query using discrete Frechet distance, leveraging IPPE and bilinear pairings for trajectory privacy protection and employing dual R-trees for trajectory filtering.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 17,2026 at 03:39:16 UTC from IEEE Xplore. Restrictions apply.

The above encryption-based methods primarily focus on similarity range queries of trajectories, but users often face difficulty in selecting an appropriate query threshold to obtain results with the desired scale. In addition, these existing schemes mainly consider the spatial features of trajectory similarity, while ignoring the temporal features that are also crucial for capturing motion patterns. In contrast, our privacy-preserving top-k and range queries schemes integrate space-filling encoding and encryption techniques, offering an effective solution for spatio-temporal similarity queries over trajectory data. This approach achieves a balance between query precision and efficiency while protecting privacy.

VIII. Conclusion

In this paper, we propose privacy-preserving trajectory similarity query schemes for top-k and range queries, employing a spatio-temporal similarity measure, DSED, as the evaluation metric. To enhance query efficiency, we design a filtration method that leverages the location-preserving property of the Hilbert curve, integrating multiple Hilbert curves and pivot points. Based on this, most dissimilar trajectories can be excluded while preserving data privacy, and the candidate trajectories are subsequently refined to the final query results through homomorphic encryption. Security analysis confirms that our schemes protect the privacy of the query request, query results, and DO's trajectory data. Experimental results indicate that our Hilbert-curve-based filtration method filters out over 90% trajectories primarily, and the proposed trajectory similarity query schemes for top-k and range queries achieve an order of magnitude higher computational efficiency compared with existing methods.

References

[1] K. Yi et al., "Towards efficient privacy-preserving top-K trajectory similarity query," in *Proc. IEEE 20th Int. Conf. Mobile Ad Hoc Smart Syst.*, 2023, pp. 512–520.

[2] O. Abul, F. Bonchi, and M. Nanni, "Never walk alone: Uncertainty for anonymity in moving objects databases," in *Proc. IEEE Int. Conf. Data Eng.*, 2008, pp. 376–385.

[3] H. Wang, Y. Li, C. Gao, G. Wang, X. Tao, and D. Jin, "Anonymization and de-anonymization of mobility trajectories: Dissecting the gaps between theory and practice," *IEEE Trans. Mobile Comput.*, vol. 20, no. 3, pp. 796–815, Mar. 2021.

[4] J. Hua, Y. Gao, and S. Zhong, "Differentially private publication of general time-serial trajectory data," in *Proc. IEEE Conf. Comput. Commun.*, 2015, pp. 549–557.

[5] M. E. Gursoy, L. Liu, S. Truex, and L. Yu, "Differentially private and utility preserving publication of trajectory data," *IEEE Trans. Mobile Comput.*, vol. 18, no. 10, pp. 2315–2329, Oct. 2019.

[6] A. Liu, Z. Kai, L. Lu, G. Liu, Z. Lei, and X. Zhou, "Efficient secure similarity computation on encrypted trajectory data," in *Proc. IEEE Int. Conf. Data Eng.*, 2015, pp. 66–77.

[7] Y. Teng, Z. Shi, F. Zhao, G. Ding, L. Xu, and C. Fan, "Signature-based secure trajectory similarity search," in *Proc. IEEE 20th Int. Conf. Trust, Secur. Privacy Comput. Commun.*, 2021, pp. 196–206.

[8] Y. Guan, R. Lu, Y. Zheng, S. Zhang, J. Shao, and G. Wei, "Achieving privacy-preserving discrete fréchet distance range queries," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 3, pp. 2097–2110, May/Jun. 2023.

[9] H. Su, S. Liu, B. Zheng, X. Zhou, and K. Zheng, "A survey of trajectory distance measures and performance evaluation," *VLDB J.*, vol. 29, no. 1, pp. 3–32, Oct. 2019.

[10] A. R. Butz, "Space filling curves and mathematical programming," *Inf. Control*, vol. 12, no. 4, pp. 314–330, 1968.

[11] Z. He and A. B. Owen, "Extensible grids: Uniform sampling on a space filling curve," *J. Roy. Stat. Soc. Ser. B, Stat. Methodol.*, vol. 78, no. 4, pp. 917–931, 2015.

[12] A. Khoshgozaran, H. Shirani-Mehr, and C. Shahabi, "Blind evaluation of location based queries using space transformation to preserve location privacy," *GeoInformatica*, vol. 17, no. 4, pp. 599–634, 2013.

[13] S. Zhang, S. Ray, R. Lu, Y. Zheng, Y. Guan, and J. Shao, "PPAQ: Privacy-preserving aggregate queries for optimal location selection in road networks," *IEEE Internet Things J.*, vol. 9, no. 20, pp. 20178–20188, Oct. 2022.

[14] S. Zhang, S. Ray, R. Lu, Y. Zheng, Y. Guan, and J. Shao, "Achieving efficient and privacy-preserving dynamic skyline query in online medical diagnosis," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9973–9986, Jun. 2022.

[15] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Proc. 23rd Int. Conf. Theory Appl. Cryptol. Inf. Secur., Adv. Cryptol.*, 2017, pp. 409–437.

[16] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," *J. ACM*, vol. 60, pp. 1–35, 2010.

[17] Z. Qin, H. Xiong, S. Wu, and J. Batamuliza, "A survey of proxy re-encryption for secure data sharing in cloud computing," *IEEE Trans. Serv. Comput.*, early access, Apr. 06, 2016, doi: 10.1109/TSC.2016.2551238.

[18] Y. Zheng, R. Lu, and J. Shao, "Achieving efficient and privacy-preserving k-NN query for outsourced ehealthcare data," *J. Med. Syst.*, vol. 43, no. 5, pp. 1–13, May 2019.

[19] B. Moon, H. V. Jagadish, C. Faloutsos, and J. H. Saltz, "Analysis of the clustering properties of the Hilbert space-filling curve," *IEEE Trans. Knowl. Data Eng.*, vol. 13, no. 1, pp. 124–141, Jan./Feb. 2001.

[20] D. J. Weston, "Exploiting novel properties of space-filling curves for data analysis," in *Proc. 13th Int. Symp., Adv. Intell. Data Anal.* , 2014, pp. 356–367.

[21] J. Wood, "Filter and refine strategy," in *Encyclopedia of GIS*. Berlin, Germany: Springer, 2008, pp. 320.

[22] Y. Zheng, "Trajectory data mining: An overview," *ACM Trans. Intell. Syst. Technol.*, vol. 6, no. 3, May 2015, Art. no. 29.

[23] J. H. Cheon, D. Kim, and D. Kim, "Efficient homomorphic comparison methods with optimal complexity," in *Proc. 26th Int. Conf. Theory Appl. Cryptology Inform. Secur. Adv. Cryptology–ASIACRYPT*, 2020, pp. 221–256.

[24] J. Yuan et al., "T-drive: Driving directions based on taxi trajectories," in *Proc. 18th SIGSPATIAL Int. Conf. Adv. Geographic Inf. Syst.*, 2010, pp. 99–108.

[25] Y. Zheng, X. Xie, and W. -Y. Ma, "Geolife2.0: A location-based social networking service," in *Proc. 10th Int. Conf. Mobile Data Manage.*, May 2009, pp. 357–358.

[26] L. J. Guibas, J. S. B. Mitchell, and T. Roos, "Voronoi diagrams of moving points in the plane," in *Proc. 17th Int. Workshop, Graph-Theoretic Concepts Comput. Sci.*, 1992, pp. 113–125.

[27] E. Frentzos, K. Gratsias, and Y. Theodoridis, "Index-based most similar trajectory search," in *Proc. IEEE 23rd Int. Conf. Data Eng.*, 2007, pp. 816–825.

[28] D. Zhang, Z. Chang, S. Wu, Y. Yuan, K. -L. Tan, and G. Chen, "Continuous trajectory similarity search for online outlier detection," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 10, pp. 4690–4704, Oct. 2022.

[29] M. Müller, *Information Retrieval for Music and Motion*, vol. 2. Berlin, Germany: Springer, 2007.

[30] M. T. Robinson, "The temporal development of collision cascades in the binary-collision approximation," *Nucl. Instrum. Methods Phys. Res. Sect. B, Beam Interact. Mater. At.*, vol. 48, no. 1-4, pp. 408–413, 1990.

[31] S. Shang, R. Ding, K. Zheng, C. S. Jensen, P. Kalnis, and X. Zhou, "Personalized trajectory matching in spatial networks," *VLDB J.*, vol. 23, pp. 449–468, 2014.

[32] Z. Ye, X. Wang, Z. Liu, D. Zhu, and J. Ma, "OBIR-tree: An efficient oblivious index for spatial keyword queries on secure enclaves," *Proc. ACM Manage. Data*, vol. 3, no. 1, pp. 1–24, 2025.

[33] X. Wang, J. Ma, X. Liu, Y. Miao, Y. Liu, and R. H. Deng, "Forward/backward and content private DSSE for spatial keyword queries," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 4, pp. 3358–3370, Jul./Aug. 2023.

[34] X. Wang, J. Ma, F. Li, X. Liu, Y. Miao, and R. H. Deng, "Enabling efficient spatial keyword queries on encrypted data with strong security guarantees," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 4909–4923, 2021.

[35] X. Wang et al., "Search me in the dark: Privacy-preserving boolean range query over encrypted spatial data," in *Proc. IEEE INFOCOM 2020 - IEEE Conf. Comput. Commun.*, 2020, pp. 2253–2262.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 17,2026 at 03:39:16 UTC from IEEE Xplore. Restrictions apply.